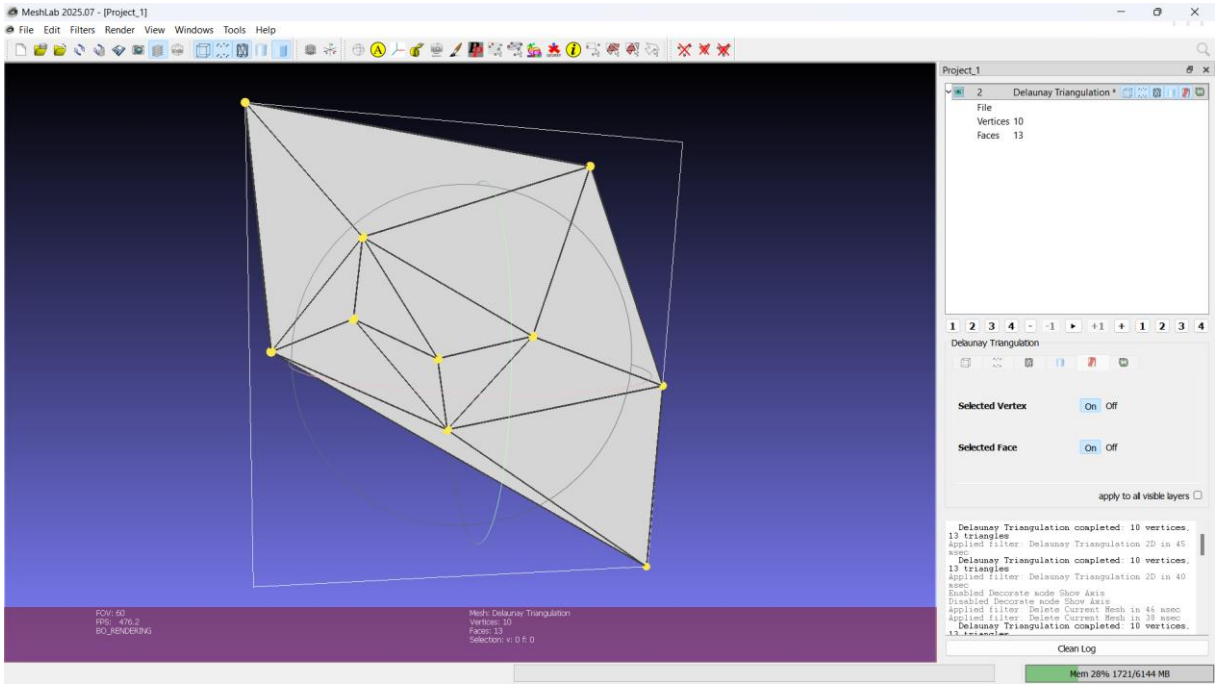
Course: Advanced Computer Graphics

Instructor: Assoc. Prof. Nguyen Van Sinh
Email: nvsinh@hcmiu.edu.vn

Assignment 1

Name: Truong Tri Dung
ID: MITIU25208

Delaunay Triangulation Algorithm for triangulation of a given 2D points



This MeshLab plugin implements the Bowyer-Watson algorithm for 2D Delaunay Triangulation.

The plugin consists of 3 files: CMakeLists.txt (build config using add_meshlab_plugin), filter_delaunay.h (class definition with Triangle and Edge structs), and filter_delaunay.cpp (implementation).

The core algorithm in delaunayTriangulation2D() works as follows:

(1) Generate N random 2D points with z=0 using srand/rand.

(2) Create a super-triangle with vertices at (-margin, -margin), (scale+3*margin, -margin), (-margin, scale+3*margin) that encloses all points.

(3) For each point, call isInsideCircumcircle() which computes the circumcenter (ux, uy) using the formula D = 2*(ax*(by-cy) + bx*(cy-ay) + cx*(ay-by)), then checks if distance^2 < radius^2.

(4) Collect boundary edges (edges not shared between two bad triangles) using sorted indices (smin, smax) for comparison while preserving original vertex order (v0, v1) for correct winding.

(5) Remove bad triangles and create new ones connecting boundary edges to the inserted point.

(6) After all points inserted, remove triangles sharing vertices with the super-triangle.

(7) Build the MeshLab mesh using tri::Allocator<CMeshO>::AddVertices and AddFaces, then update with tri::UpdateBounding::Box and tri::UpdateNormal::PerVertexNormalizedPerFaceNormalized.

Source code

CMakeLists.txt

```
# Copyright 2019-2020, Collabora, Ltd.
# SPDX-License-Identifier: BSL-1.0

set(SOURCES filter_delaunay.cpp)

set(HEADERS filter_delaunay.h)

add_meshlab_plugin(filter_delaunay
${SOURCES} ${HEADERS})
```

filter_delaunay.h

```
#ifndef FILTER_DELAUNAY_H
#define FILTER_DELAUNAY_H

#include
<common/plugins/interfaces/filter_plugin.h
>

class FilterDelaunayPlugin : public
QObject, public FilterPlugin
{
   Q_OBJECT
   MESHLAB_PLUGIN_IID_EXPORTER(
FILTER_PLUGIN_IID)
   Q_INTERFACES(FilterPlugin)

public:
   enum { FP_DELAUNAY_2D };

   FilterDelaunayPlugin();
   virtual ~FilterDelaunayPlugin();

   QString pluginName() const;
   QString filterName(ActionIDType filter)
const;
   QString pythonFilterName(ActionIDType
f) const;
   QString filterInfo(ActionIDType filter)
const;
   FilterClass getClass(const QAction* a)
const;
   FilterArity filterArity(const QAction*)
const;
   int getPreConditions(const QAction *)
const;
   int postCondition(const QAction*) const;
   RichParameterList
initParameterList(const QAction*, const
MeshModel &m);
   std::map<std::string, QVariant>
applyFilter(
         const QAction* action,
         const RichParameterList &
parameters,
         MeshDocument &md,
         unsigned int& postConditionMask,
         vcg::CallBackPos * cb);

private:
   // Bowyer-Watson Delaunay triangulation
   struct Triangle {
      int v0, v1, v2;
      Triangle(int a, int b, int c) : v0(a),
v1(b), v2(c) {}
   };

   struct Edge {
      int v0, v1;       // original order (for
building new triangles)
      int smin, smax;    // sorted order (for
comparison)
      Edge(int a, int b) : v0(a), v1(b),
smin(std::min(a,b)), smax(std::max(a,b)) {}
```

```
        bool operator==(const Edge& other) const {
            return smin == other.smin && smax == other.smax;
        }
    };

    bool isInsideCircumcircle(
        const std::vector<Point3m>& points,
        const Triangle& tri,
        const Point3m& p) const;

    bool delaunayTriangulation2D(
            MeshDocument &md,
            vcg::CallBackPos *cb,
            int numPoints,
            int randomSeed,
            Scalarm scale);
};

#endif
```

filter_delaunay.cpp

```
#include "filter_delaunay.h"

#include <vcg/complex/algorithms/update/bounding.h>
#include <vcg/complex/algorithms/update/normal.h>
#include <vcg/complex/algorithms/update/topology.h>

#include <vector>
#include <algorithm>
#include <cmath>
#include <cstdlib>
#include <ctime>

using namespace vcg;

//
==========================================================================
// Constructor
//
==========================================================================
FilterDelaunayPlugin::FilterDelaunayPlugin()
{
    typeList = {FP_DELAUNAY_2D};

    for (ActionIDType tt : types())
        actionList.push_back(new QAction(filterName(tt), this));
}

FilterDelaunayPlugin::~FilterDelaunayPlugin()
{
}

//
==========================================================================
// Plugin metadata
//
==========================================================================
QString FilterDelaunayPlugin::pluginName() const
{
    return "FilterDelaunay";
}

QString FilterDelaunayPlugin::filterName(ActionIDType filterId) const
{
    switch (filterId) {
    case FP_DELAUNAY_2D:
        return "Delaunay Triangulation 2D";
    default:
        assert(0);
        return QString();
    }
}

QString FilterDelaunayPlugin::pythonFilterName(ActionIDType f) const
{
    switch (f) {
    case FP_DELAUNAY_2D:
        return "delaunay_triangulation_2d";
    default:
        assert(0);
        return QString();
    }
```

```
}

QString
FilterDelaunayPlugin::filterInfo(ActionIDT
ype filterId) const
{
   switch (filterId) {
   case FP_DELAUNAY_2D:
      return "Generate a 2D Delaunay
Triangulation from random points using the
"
            "Bowyer-Watson algorithm.
Creates a new mesh where vertices are
random "
            "2D points and faces are the
Delaunay triangles. The Delaunay "
            "triangulation maximizes the
minimum angle of all triangles, avoiding "
            "skinny triangles.";
   default:
      assert(0);
      return QString();
   }
}

FilterDelaunayPlugin::FilterClass
FilterDelaunayPlugin::getClass(const
QAction *a) const
{
   switch (ID(a)) {
   case FP_DELAUNAY_2D:
      return FilterPlugin::MeshCreation;
   default:
      assert(0);
      return FilterPlugin::Generic;
   }
}

FilterPlugin::FilterArity
FilterDelaunayPlugin::filterArity(const
QAction *) const
{
   return NONE;
}

int
FilterDelaunayPlugin::getPreConditions(con
st QAction *) const
{
   return MeshModel::MM_NONE;
}

int
FilterDelaunayPlugin::postCondition(const
QAction *) const
{
   return MeshModel::MM_VERTCOORD |
MeshModel::MM_FACENORMAL |
MeshModel::MM_VERTNORMAL;
}

//
==========================
==========================
========
// Parameters
//
==========================
==========================
========
RichParameterList
FilterDelaunayPlugin::initParameterList(con
st QAction *action, const MeshModel &)
{
   RichParameterList parlst;
   switch (ID(action)) {
   case FP_DELAUNAY_2D:
      parlst.addParam(RichInt(
         "NumPoints", 10, "Number of
Points",
         "Number of random 2D points to
generate (recommended: 10 to 15)."));
      parlst.addParam(RichInt(
         "RandomSeed", 42, "Random Seed",
         "Seed for random number generator.
Use 0 for a time-based seed."));
      parlst.addParam(RichFloat(
         "Scale", 100.0f, "Scale",
         "Scale factor for the random point
positions."));
      break;
   default:
      assert(0);
   }
   return parlst;
}

//
==========================
==========================
========
// Apply filter
//
==========================
```

```
==========================
========
std::map<std::string, QVariant>
FilterDelaunayPlugin::applyFilter(
   const QAction *filter,
   const RichParameterList &parameters,
   MeshDocument &md,
   unsigned int &,
   vcg::CallBackPos *cb)
{
   switch (ID(filter)) {
   case FP_DELAUNAY_2D:
      delaunayTriangulation2D(
         md, cb,
         parameters.getInt("NumPoints"),
         parameters.getInt("RandomSeed"),
         parameters.getFloat("Scale"));
      break;
   default:
      wrongActionCalled(filter);
   }
   return std::map<std::string, QVariant>();
}

//
==========================
==========================
========
// Circumcircle test for Bowyer-Watson
// Computes circumcircle center and radius,
then checks distance.
// Works regardless of triangle winding
order.
//
==========================
==========================
========
bool
FilterDelaunayPlugin::isInsideCircumcircle(
   const std::vector<Point3m> &points,
   const Triangle &tri,
   const Point3m &p) const
{
   Scalarm ax = points[tri.v0].X();
   Scalarm ay = points[tri.v0].Y();
   Scalarm bx = points[tri.v1].X();
   Scalarm by = points[tri.v1].Y();
   Scalarm cx = points[tri.v2].X();
   Scalarm cy = points[tri.v2].Y();

   Scalarm D = 2.0 * (ax * (by - cy) + bx *
(cy - ay) + cx * (ay - by));
   if (std::abs(D) < 1e-12)
      return false; // degenerate triangle

   Scalarm ux = ((ax * ax + ay * ay) * (by -
cy) + (bx * bx + by * by) * (cy - ay) + (cx *
cx + cy * cy) * (ay - by)) / D;
   Scalarm uy = ((ax * ax + ay * ay) * (cx -
bx) + (bx * bx + by * by) * (ax - cx) + (cx *
cx + cy * cy) * (bx - ax)) / D;

   Scalarm r2 = (ax - ux) * (ax - ux) + (ay -
uy) * (ay - uy);
   Scalarm d2 = (p.X() - ux) * (p.X() - ux) +
(p.Y() - uy) * (p.Y() - uy);

   return d2 < r2;
}

//
==========================
==========================
========
// Bowyer-Watson Delaunay Triangulation
//
==========================
==========================
========
bool
FilterDelaunayPlugin::delaunayTriangulatio
n2D(
   MeshDocument &md,
   vcg::CallBackPos *cb,
   int numPoints,
   int randomSeed,
   Scalarm scale)
{
   if (numPoints < 3) {
      log("Error: Need at least 3 points for
triangulation");
      return false;
   }

   // Seed random number generator
   if (randomSeed == 0)
      srand((unsigned int)time(NULL));
   else
      srand((unsigned int)randomSeed);

   cb(0, "Generating random points...");

   // Step 1: Generate random 2D points (z
= 0)
```

```
    std::vector<Point3m> points;

    for (int i = 0; i < numPoints; i++) {
        Scalarm x = (Scalarm(rand()) /
RAND_MAX) * scale;
        Scalarm y = (Scalarm(rand()) /
RAND_MAX) * scale;
        points.push_back(Point3m(x, y, 0));
    }

    cb(10, "Creating super-triangle...");

    // Step 2: Create super-triangle that
contains all points
    Scalarm margin = scale * 10.0;
    int st0 = (int)points.size();
    int st1 = st0 + 1;
    int st2 = st0 + 2;
    points.push_back(Point3m(-
margin,          -margin,          0));
    points.push_back(Point3m(scale + margin
* 3, -margin,          0));
    points.push_back(Point3m(-
margin,          scale + margin * 3, 0));

    std::vector<Triangle> triangles;
    triangles.push_back(Triangle(st0, st1,
st2));

    cb(20, "Running Bowyer-Watson
algorithm...");

    // Step 3: Insert each point using Bowyer-
Watson
    for (int i = 0; i < numPoints; i++) {
        cb(20 + 60 * i / numPoints, "Inserting
points...");

        // Find all triangles whose circumcircle
contains the point
        std::vector<Triangle> badTriangles;
        for (size_t t = 0; t < triangles.size();
t++) {
            if (isInsideCircumcircle(points,
triangles[t], points[i])) {
                badTriangles.push_back(triangles[
t]);
            }
        }

        // Find the boundary polygon (edges
not shared by two bad triangles)
        std::vector<Edge> boundary;
        for (size_t t = 0; t <
badTriangles.size(); t++) {
            Edge edges[3] = {
                Edge(badTriangles[t].v0,
badTriangles[t].v1),
                Edge(badTriangles[t].v1,
badTriangles[t].v2),
                Edge(badTriangles[t].v2,
badTriangles[t].v0)
            };

            for (int e = 0; e < 3; e++) {
                bool shared = false;
                for (size_t t2 = 0; t2 <
badTriangles.size(); t2++) {
                    if (t2 == t) continue;
                    Edge otherEdges[3] = {
                        Edge(badTriangles[t2].v0,
badTriangles[t2].v1),
                        Edge(badTriangles[t2].v1,
badTriangles[t2].v2),
                        Edge(badTriangles[t2].v2,
badTriangles[t2].v0)
                    };
                    for (int e2 = 0; e2 < 3; e2++) {
                        if (edges[e] ==
otherEdges[e2]) {
                            shared = true;
                            break;
                        }
                    }
                    if (shared) break;
                }
                if (!shared) {
                    boundary.push_back(edges[e]);
                }
            }
        }

        // Remove bad triangles
        std::vector<Triangle> newTriangles;
        for (size_t t = 0; t < triangles.size();
t++) {
            bool isBad = false;
            for (size_t b = 0; b <
badTriangles.size(); b++) {
                if (triangles[t].v0 ==
badTriangles[b].v0 &&
                    triangles[t].v1 ==
badTriangles[b].v1 &&
```

```
                triangles[t].v2 ==
badTriangles[b].v2) {
                isBad = true;
                break;
            }
        }
        if (!isBad) {
            newTriangles.push_back(triangles
[t]);
        }
    }
    triangles = newTriangles;

    // Create new triangles from boundary
edges to the new point
    for (size_t e = 0; e < boundary.size();
e++) {
        triangles.push_back(Triangle(bound
ary[e].v0, boundary[e].v1, i));
    }
  }

  cb(80, "Removing super-triangle...");

  // Step 4: Remove triangles that share
vertices with the super-triangle
  std::vector<Triangle> finalTriangles;
  for (size_t t = 0; t < triangles.size(); t++) {
    if (triangles[t].v0 != st0 &&
triangles[t].v0 != st1 && triangles[t].v0 !=
st2 &&
        triangles[t].v1 != st0 &&
triangles[t].v1 != st1 && triangles[t].v1 !=
st2 &&
        triangles[t].v2 != st0 &&
triangles[t].v2 != st1 && triangles[t].v2 !=
st2) {
        finalTriangles.push_back(triangles[t]
);
    }
  }

  cb(90, "Building mesh...");

  // Step 5: Create MeshLab mesh from the
triangulation
  MeshModel *mm = md.addNewMesh("",
"Delaunay Triangulation");
  CMeshO &mesh = mm->cm;

  // Add vertices (only the original points,
not the super-triangle)
  auto vi =
tri::Allocator<CMeshO>::AddVertices(mes
h, numPoints);
  for (int i = 0; i < numPoints; i++) {
    vi[i].P() = points[i];
  }

  // Validate face indices before adding
  int validFaces = 0;
  for (size_t i = 0; i < finalTriangles.size();
i++) {
    if (finalTriangles[i].v0 >= 0 &&
finalTriangles[i].v0 < numPoints &&
        finalTriangles[i].v1 >= 0 &&
finalTriangles[i].v1 < numPoints &&
        finalTriangles[i].v2 >= 0 &&
finalTriangles[i].v2 < numPoints) {
        validFaces++;
    }
  }

  // Add only valid faces
  auto fi =
tri::Allocator<CMeshO>::AddFaces(mesh,
validFaces);
  int fIdx = 0;
  for (size_t i = 0; i < finalTriangles.size();
i++) {
    if (finalTriangles[i].v0 >= 0 &&
finalTriangles[i].v0 < numPoints &&
        finalTriangles[i].v1 >= 0 &&
finalTriangles[i].v1 < numPoints &&
        finalTriangles[i].v2 >= 0 &&
finalTriangles[i].v2 < numPoints) {
        fi[fIdx].V(0) =
&mesh.vert[finalTriangles[i].v0];
        fi[fIdx].V(1) =
&mesh.vert[finalTriangles[i].v1];
        fi[fIdx].V(2) =
&mesh.vert[finalTriangles[i].v2];
        fIdx++;
    }
  }

  // Update mesh properties
  tri::UpdateBounding<CMeshO>::Box(me
sh);
  tri::UpdateNormal<CMeshO>::PerVertex
NormalizedPerFaceNormalized(mesh);

  log("Delaunay Triangulation
completed: %i vertices, %i triangles",
```

```
    mesh.vn, mesh.fn);

cb(100, "Done!");

return true;
```

```
}

MESHLAB_PLUGIN_NAME_EXPORTER(FilterDelaunayPlugin)
```

Lab 3: Advance graphic in 2D, Transformation, Animation

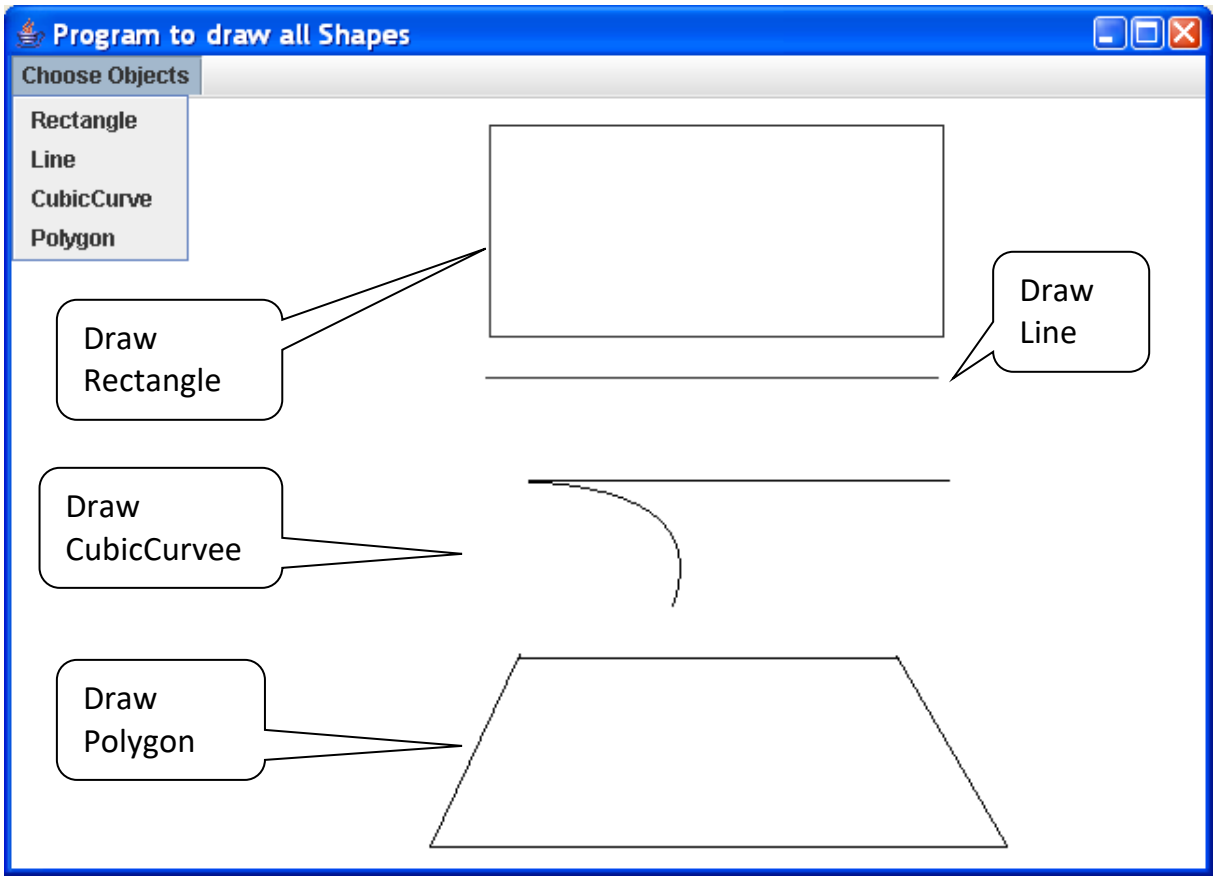
Contents:

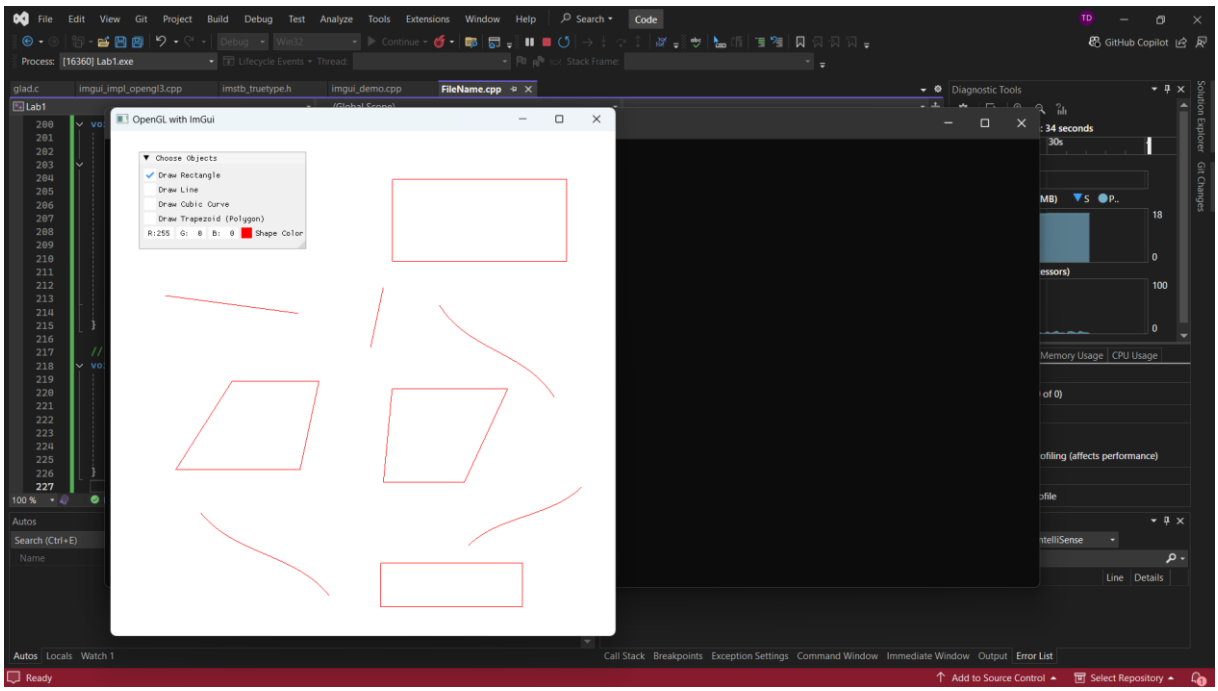
- **Advance graphic in 2D**
- **Transformation**
- **Animation**
- **Exercises**

Duration: 3 hours

Part 1: Advance graphic in 2D

In this part, we will create a application "Program to draw all Shapes". There is a menu "Choose Objects", when user click here, she/he can choose one of the objects in the dropdown menu to draw.





This program uses OpenGL (GLEW + GLFW) with ImGui for GUI. It defines a Shape struct with types {RECTANGLE, LINE, CUBIC_CURVE, POLYGON} and stores drawn shapes in std::vector<Shape>.

Mouse input: on left-button press, records (startX, startY) normalized to [-1,1]; on release, records (endX, endY) and pushes a Shape.

Drawing functions: drawRectangle() uses GL_LINE_LOOP with 4 vertices; drawLine() uses GL_LINES with 2 vertices; drawCubicCurve() uses GL_LINE_STRIP with Bezier interpolation B(t) = (1-t)^3*P0 + 3(1-t)^2*t*P1 + 3(1-t)*t^2*P2 + t^3*P3; drawPolygon() uses GL_POLYGON to draw a trapezoid with 4 computed vertices.

ImGui panel provides checkboxes (drawRectangleFlag, drawLineFlag, etc.) and ImGui::ColorEdit3 for color selection.

```
//Truong Tri Dung – MITIU25208
#define GLEW_STATIC
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include "imgui.h"
#include "imgui_impl_glfw.h"
#include "imgui_impl_opengl3.h"

#include <iostream>
#include <vector>

// Structure to store shape data
struct Shape {
    enum Type { RECTANGLE, LINE, CUBIC_CURVE, POLYGON } type;
    float x1, y1, x2, y2;
    ImVec4 color;
};

// Function declarations for drawing shapes
void drawRectangle(float x1, float y1, float x2, float y2, const ImVec4& color);
void drawLine(float x1, float y1, float x2, float y2, const ImVec4& color);
void drawCubicCurve(float x1, float y1, float x2, float y2, const ImVec4& color);
void drawPolygon(float x1, float y1, float x2, float y2, const ImVec4& color);
```

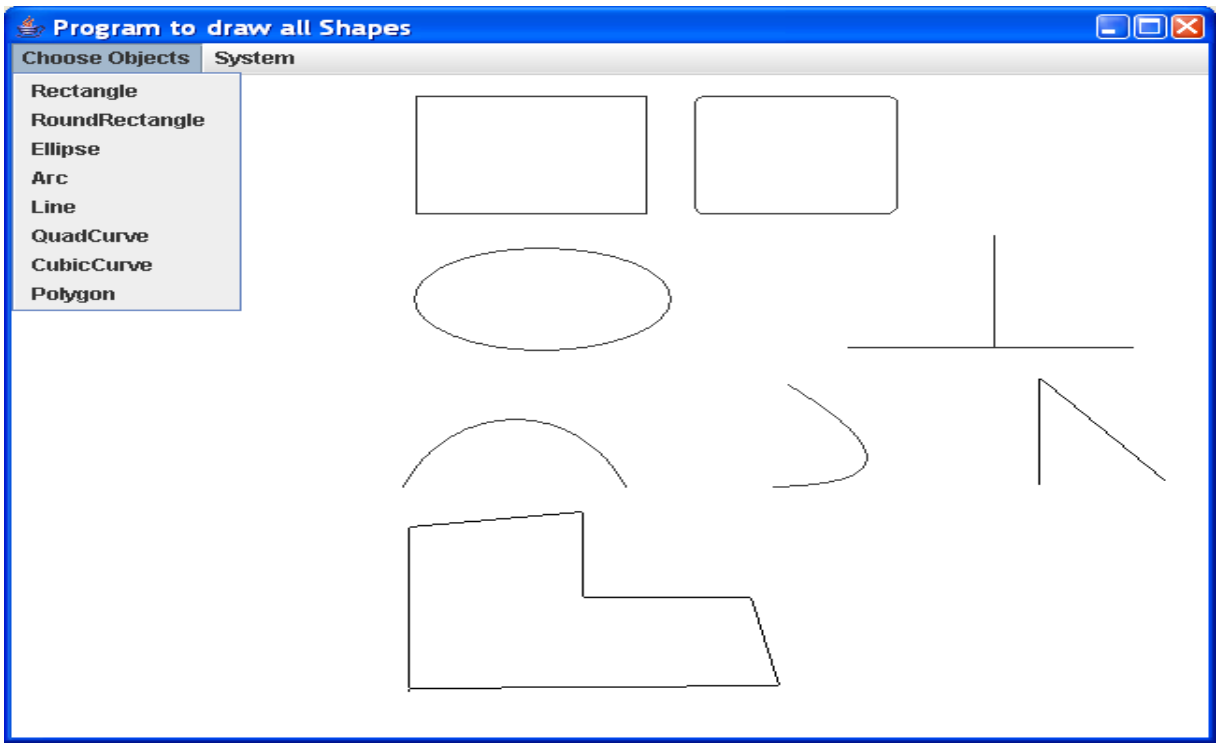
```
void processInput(GLFWwindow*
window);

const int WINDOW_WIDTH = 800;
const int WINDOW_HEIGHT = 800;

// Global variables for mouse input
bool isDrawing = false;
float startX, startY, endX, endY;
std::vector<Shape> shapes;

int main() {
   if (!glfwInit()) {
      std::cout << "Failed to initialize
GLFW" << std::endl;
      return -1;
   }

   GLFWwindow* window =
glfwCreateWindow(WINDOW_WIDTH,
WINDOW_HEIGHT, "OpenGL with
ImGui", NULL, NULL);
   if (!window) {
      std::cout << "Failed to create GLFW
window" << std::endl;
      glfwTerminate();
      return -1;
   }

   glfwMakeContextCurrent(window);
   glewExperimental = true;
   if (glewInit() != GLEW_OK) {
      std::cout << "Failed to initialize
GLEW" << std::endl;
      return -1;
   }

   glViewport(0, 0, WINDOW_WIDTH,
WINDOW_HEIGHT);

   // Initialize ImGui
   IMGUI_CHECKVERSION();
   ImGui::CreateContext();
   ImGuiIO& io = ImGui::GetIO(); (void)io;

   ImGui::StyleColorsLight();


ImGui_ImplGlfw_InitForOpenGL(window,
true);
   ImGui_ImplOpenGL3_Init("#version
330");

   // Variables for checkboxes, size, and
color
   bool drawRectangleFlag = false;
   bool drawLineFlag = false;
   bool drawCubicCurveFlag = false;
   bool drawPolygonFlag = false;
   ImVec4 shapeColor = ImVec4(1.0f, 0.0f,
0.0f, 1.0f);  // Red by default

   // Main loop
   while
(!glfwWindowShouldClose(window)) {
      processInput(window);

      // Start the ImGui frame
      ImGui_ImplOpenGL3_NewFrame();
      ImGui_ImplGlfw_NewFrame();
      ImGui::NewFrame();

      // ImGui window
      ImGui::Begin("Choose Objects");
      ImGui::Checkbox("Draw Rectangle",
&drawRectangleFlag);
      ImGui::Checkbox("Draw Line",
&drawLineFlag);
      ImGui::Checkbox("Draw Cubic
Curve", &drawCubicCurveFlag);
      ImGui::Checkbox("Draw Trapezoid
(Polygon)", &drawPolygonFlag);
      ImGui::ColorEdit3("Shape Color",
(float*)&shapeColor);
      ImGui::End();

      // Handle mouse input for drawing
shapes
      if (glfwGetMouseButton(window,
GLFW_MOUSE_BUTTON_LEFT) ==
GLFW_PRESS) {
         if (!isDrawing) {
            isDrawing = true;
            double mouseX, mouseY;
            glfwGetCursorPos(window,
&mouseX, &mouseY);
            startX = (mouseX /
WINDOW_WIDTH) * 2 - 1; // Normalize
to OpenGL coordinates
            startY = 1 - (mouseY /
WINDOW_HEIGHT) * 2;
         }
         else {
            double mouseX, mouseY;
```

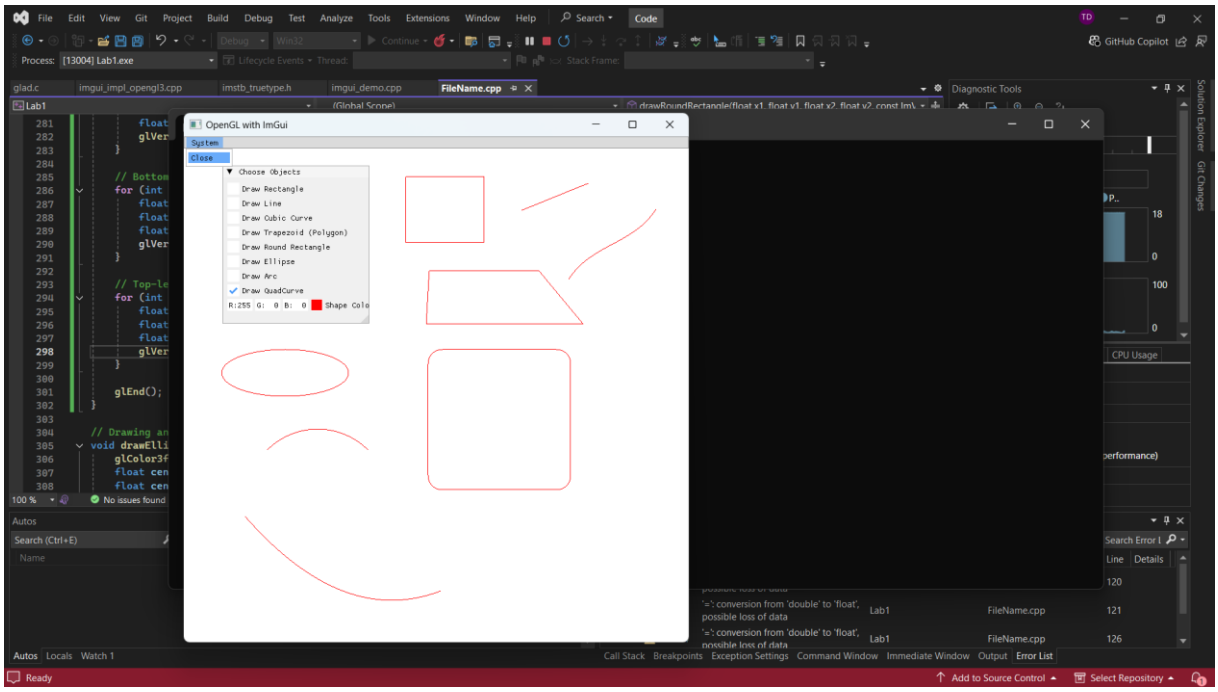
```
            glfwGetCursorPos(window,
&mouseX, &mouseY);
            endX = (mouseX /
WINDOW_WIDTH) * 2 - 1;
            endY = 1 - (mouseY /
WINDOW_HEIGHT) * 2;
        }
    }
    else if (isDrawing &&
glfwGetMouseButton(window,
GLFW_MOUSE_BUTTON_LEFT) ==
GLFW_RELEASE) {
        // Add the shape to the list when
mouse is released
        Shape shape;
        if (drawRectangleFlag) {
            shape.type =
Shape::RECTANGLE;
        }
        else if (drawLineFlag) {
            shape.type = Shape::LINE;
        }
        else if (drawCubicCurveFlag) {
            shape.type =
Shape::CUBIC_CURVE;
        }
        else if (drawPolygonFlag) {
            shape.type = Shape::POLYGON;
        }
        shape.x1 = startX;
        shape.y1 = startY;
        shape.x2 = endX;
        shape.y2 = endY;
        shape.color = shapeColor;
        shapes.push_back(shape);
        isDrawing = false;
    }

    // Rendering OpenGL
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    // Draw all shapes
    for (const Shape& shape : shapes) {
        switch (shape.type) {
        case Shape::RECTANGLE:
            drawRectangle(shape.x1,
shape.y1, shape.x2, shape.y2, shape.color);
            break;
        case Shape::LINE:
            drawLine(shape.x1, shape.y1,
shape.x2, shape.y2, shape.color);
            break;
        case Shape::CUBIC_CURVE:
            drawCubicCurve(shape.x1,
shape.y1, shape.x2, shape.y2, shape.color);
            break;
        case Shape::POLYGON:
            drawPolygon(shape.x1, shape.y1,
shape.x2, shape.y2, shape.color);
            break;
        }
    }

    // Render ImGui
    ImGui::Render();

ImGui_ImplOpenGL3_RenderDrawData(Im
Gui::GetDrawData());

    // Swap buffers and poll events
    glfwSwapBuffers(window);
    glfwPollEvents();
  }

  // Cleanup
  ImGui_ImplOpenGL3_Shutdown();
  ImGui_ImplGlfw_Shutdown();
  ImGui::DestroyContext();

  glfwDestroyWindow(window);
  glfwTerminate();

  return 0;
}

// Function to handle input
void processInput(GLFWwindow* window)
{
  if (glfwGetKey(window,
GLFW_KEY_ESCAPE) ==
GLFW_PRESS) {
    glfwSetWindowShouldClose(window,
true);
  }
}

// Drawing functions for shapes
void drawRectangle(float x1, float y1, float
x2, float y2, const ImVec4& color) {
  glColor3f(color.x, color.y, color.z);
  glBegin(GL_LINE_LOOP);
  glVertex2f(x1, y1);
  glVertex2f(x2, y1);
```

```
    glVertex2f(x2, y2);
    glVertex2f(x1, y2);
    glEnd();
}

void drawLine(float x1, float y1, float x2,
float y2, const ImVec4& color) {
    glColor3f(color.x, color.y, color.z);
    glBegin(GL_LINES);
    glVertex2f(x1, y1);
    glVertex2f(x2, y2);
    glEnd();
}

// Drawing a cubic Bézier curve between
two points
void drawCubicCurve(float x1, float y1,
float x2, float y2, const ImVec4& color) {
    glColor3f(color.x, color.y, color.z);
    glBegin(GL_LINE_STRIP);
    for (float t = 0.0f; t <= 1.0f; t += 0.01f) {
        float xt = (1 - t) * (1 - t) * (1 - t) * x1
            + 3 * (1 - t) * (1 - t) * t * (x1 + (x2 -
x1) / 4)
            + 3 * (1 - t) * t * t * (x2 - (x2 - x1) /
4)
            + t * t * t * x2;
        float yt = (1 - t) * (1 - t) * (1 - t) * y1
            + 3 * (1 - t) * (1 - t) * t * (y1 + (y2 -
y1) / 2)
            + 3 * (1 - t) * t * t * (y2 - (y2 - y1) /
2)
            + t * t * t * y2;
        glVertex2f(xt, yt);
    }
    glEnd();
}

// Drawing a trapezoid between two points
void drawPolygon(float x1, float y1, float
x2, float y2, const ImVec4& color) {
    glColor3f(color.x, color.y, color.z);
    glBegin(GL_LINE_LOOP);
    glVertex2f(x1 * 0.7f, y1); // Left bottom
    glVertex2f(x2 * 0.7f, y1); // Right bottom
    glVertex2f(x2, y2); // Right top
(narrower)
    glVertex2f(x1, y2); // Left top (narrower)
    glEnd();
}
```

Exercise 1: Copy the above source code and modify:

- Add one more menu "System", when click here there is an Item "Close" to exit program.
- Add more 4 objects (RoundRectangle, Ellipse, Arc, QuadCurve) onto the menu "Choose Objects" and therefore user can draw them. The result is as below figure:





Extends the base program by adding 4 new shape types to the enum: ROUND_RECTANGLE, ELLIPSE, ARC, QUAD_CURVE.

- drawEllipse() uses GL_LINE_LOOP with parametric equations: x = cx + rx*cos(theta), y = cy + ry*sin(theta) for 100 segments.
- drawArc() draws a partial ellipse from 0 to PI using the same parametric approach with GL_LINE_STRIP.
- drawRoundRectangle() draws a rectangle with rounded corners by combining GL_LINE_STRIP straight edges with small arc segments at each corner using cos/sin.

- drawQuadCurve() uses GL_LINE_STRIP with quadratic Bezier: B(t) = (1-t)^2*P0 + 2(1-t)*t*P1 + t^2*P2, where P1 is computed as the midpoint offset.

A System > Close menu is added using ImGui::BeginMainMenuBar() with glfwSetWindowShouldClose(window, true).

```
//Truong Tri Dung - MITIU25208
#define GLEW_STATIC
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include "imgui.h"
#include "imgui_impl_glfw.h"
#include "imgui_impl_opengl3.h"

#include <iostream>
#include <vector>

// Structure to store shape data
struct Shape {
   enum Type { RECTANGLE, LINE, CUBIC_CURVE, POLYGON, ROUND_RECTANGLE, ELLIPSE, ARC, QUAD_CURVE } type;
   float x1, y1, x2, y2;
   ImVec4 color;
};

// Function declarations for drawing shapes
void drawRectangle(float x1, float y1, float x2, float y2, const ImVec4& color);
void drawLine(float x1, float y1, float x2, float y2, const ImVec4& color);
void drawCubicCurve(float x1, float y1, float x2, float y2, const ImVec4& color);
void drawPolygon(float x1, float y1, float x2, float y2, const ImVec4& color);
void drawRoundRectangle(float x1, float y1, float x2, float y2, const ImVec4& color);
void drawEllipse(float x1, float y1, float x2, float y2, const ImVec4& color);
void drawArc(float x1, float y1, float x2, float y2, const ImVec4& color);
void drawQuadCurve(float x1, float y1, float x2, float y2, const ImVec4& color);

void processInput(GLFWwindow* window);

const int WINDOW_WIDTH = 800;
const int WINDOW_HEIGHT = 800;
```

```
// Global variables for mouse input
bool isDrawing = false;
float startX, startY, endX, endY;
std::vector<Shape> shapes;

int main() {
   if (!glfwInit()) {
      std::cout << "Failed to initialize GLFW" << std::endl;
      return -1;
   }

   GLFWwindow* window = glfwCreateWindow(WINDOW_WIDTH, WINDOW_HEIGHT, "OpenGL with ImGui", NULL, NULL);
   if (!window) {
      std::cout << "Failed to create GLFW window" << std::endl;
      glfwTerminate();
      return -1;
   }

   glfwMakeContextCurrent(window);
   glewExperimental = true;
   if (glewInit() != GLEW_OK) {
      std::cout << "Failed to initialize GLEW" << std::endl;
      return -1;
   }

   glViewport(0, 0, WINDOW_WIDTH, WINDOW_HEIGHT);

   // Initialize ImGui
   IMGUI_CHECKVERSION();
   ImGui::CreateContext();
   ImGuiIO& io = ImGui::GetIO(); (void)io;

   ImGui::StyleColorsLight();


ImGui_ImplGlfw_InitForOpenGL(window, true);
```

```
    ImGui_ImplOpenGL3_Init("#version 330");

    // Variables for checkboxes, size, and color
    bool drawRectangleFlag = false;
    bool drawLineFlag = false;
    bool drawCubicCurveFlag = false;
    bool drawPolygonFlag = false;
    bool drawRoundRectangleFlag = false;
    bool drawEllipseFlag = false;
    bool drawArcFlag = false;
    bool drawQuadCurveFlag = false;
    ImVec4 shapeColor = ImVec4(1.0f, 0.0f, 0.0f, 1.0f);  // Red by default

    // Main loop
    while (!glfwWindowShouldClose(window)) {
        processInput(window);

        // Start the ImGui frame
        ImGui_ImplOpenGL3_NewFrame();
        ImGui_ImplGlfw_NewFrame();
        ImGui::NewFrame();

        // System menu with "Close" option
        if (ImGui::BeginMainMenuBar()) {
            if (ImGui::BeginMenu("System")) {
                if (ImGui::MenuItem("Close")) {
                    glfwSetWindowShouldClose(window, true);
                }
                ImGui::EndMenu();
            }
            ImGui::EndMainMenuBar();
        }

        // ImGui window for choosing objects
        ImGui::Begin("Choose Objects");
        ImGui::Checkbox("Draw Rectangle", &drawRectangleFlag);
        ImGui::Checkbox("Draw Line", &drawLineFlag);
        ImGui::Checkbox("Draw Cubic Curve", &drawCubicCurveFlag);
        ImGui::Checkbox("Draw Trapezoid (Polygon)", &drawPolygonFlag);
        ImGui::Checkbox("Draw Round Rectangle", &drawRoundRectangleFlag);
        ImGui::Checkbox("Draw Ellipse", &drawEllipseFlag);
        ImGui::Checkbox("Draw Arc", &drawArcFlag);
        ImGui::Checkbox("Draw QuadCurve", &drawQuadCurveFlag);
        ImGui::ColorEdit3("Shape Color", (float*)&shapeColor);
        ImGui::End();

        // Handle mouse input for drawing shapes
        if (glfwGetMouseButton(window, GLFW_MOUSE_BUTTON_LEFT) == GLFW_PRESS) {
            if (!isDrawing) {
                isDrawing = true;
                double mouseX, mouseY;
                glfwGetCursorPos(window, &mouseX, &mouseY);
                startX = (mouseX / WINDOW_WIDTH) * 2 - 1; // Normalize to OpenGL coordinates
                startY = 1 - (mouseY / WINDOW_HEIGHT) * 2;
            }
            else {
                double mouseX, mouseY;
                glfwGetCursorPos(window, &mouseX, &mouseY);
                endX = (mouseX / WINDOW_WIDTH) * 2 - 1;
                endY = 1 - (mouseY / WINDOW_HEIGHT) * 2;
            }
        }
        else if (isDrawing && glfwGetMouseButton(window, GLFW_MOUSE_BUTTON_LEFT) == GLFW_RELEASE) {
            // Add the shape to the list when mouse is released
            Shape shape;
            if (drawRectangleFlag) shape.type = Shape::RECTANGLE;
            if (drawLineFlag) shape.type = Shape::LINE;
            if (drawCubicCurveFlag) shape.type = Shape::CUBIC_CURVE;
            if (drawPolygonFlag) shape.type = Shape::POLYGON;
            if (drawRoundRectangleFlag) shape.type = Shape::ROUND_RECTANGLE;
```

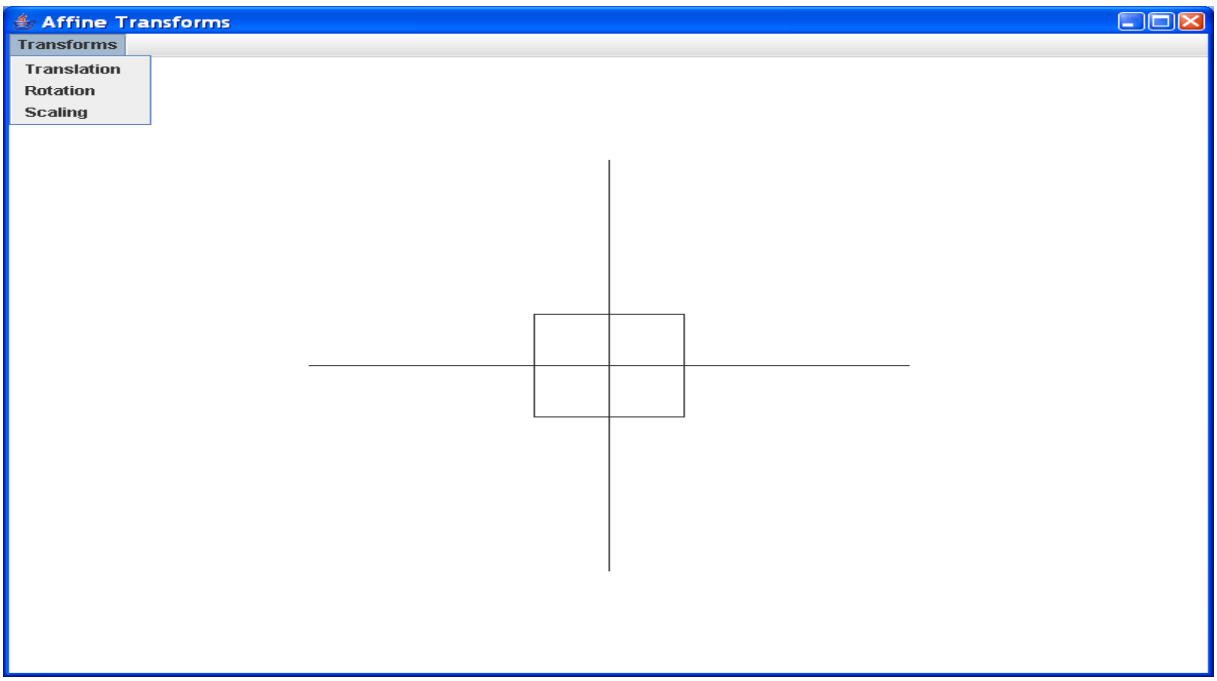
```
        if (drawEllipseFlag) shape.type = Shape::ELLIPSE;
        if (drawArcFlag) shape.type = Shape::ARC;
        if (drawQuadCurveFlag) shape.type = Shape::QUAD_CURVE;

        shape.x1 = startX;
        shape.y1 = startY;
        shape.x2 = endX;
        shape.y2 = endY;
        shape.color = shapeColor;
        shapes.push_back(shape);
        isDrawing = false;
    }

    // Rendering OpenGL
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    // Draw all shapes
    for (const Shape& shape : shapes) {
        switch (shape.type) {
        case Shape::RECTANGLE:
            drawRectangle(shape.x1, shape.y1, shape.x2, shape.y2, shape.color);
            break;
        case Shape::LINE:
            drawLine(shape.x1, shape.y1, shape.x2, shape.y2, shape.color);
            break;
        case Shape::CUBIC_CURVE:
            drawCubicCurve(shape.x1, shape.y1, shape.x2, shape.y2, shape.color);
            break;
        case Shape::POLYGON:
            drawPolygon(shape.x1, shape.y1, shape.x2, shape.y2, shape.color);
            break;
        case Shape::ROUND_RECTANGLE:
            drawRoundRectangle(shape.x1, shape.y1, shape.x2, shape.y2, shape.color);
            break;
        case Shape::ELLIPSE:
            drawEllipse(shape.x1, shape.y1, shape.x2, shape.y2, shape.color);
            break;
        case Shape::ARC:
            drawArc(shape.x1, shape.y1, shape.x2, shape.y2, shape.color);
            break;
        case Shape::QUAD_CURVE:
            drawQuadCurve(shape.x1, shape.y1, shape.x2, shape.y2, shape.color);
            break;
        }
    }

    // Render ImGui
    ImGui::Render();

ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());

    // Swap buffers and poll events
    glfwSwapBuffers(window);
    glfwPollEvents();
  }

  // Cleanup
  ImGui_ImplOpenGL3_Shutdown();
  ImGui_ImplGlfw_Shutdown();
  ImGui::DestroyContext();

  glfwDestroyWindow(window);
  glfwTerminate();

  return 0;
}

// Function to handle input
void processInput(GLFWwindow* window) {
  if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS) {
    glfwSetWindowShouldClose(window, true);
  }
}

// Drawing functions for shapes
void drawRectangle(float x1, float y1, float x2, float y2, const ImVec4& color) {
  glColor3f(color.x, color.y, color.z);
  glBegin(GL_LINE_LOOP);
  glVertex2f(x1, y1);
  glVertex2f(x2, y1);
  glVertex2f(x2, y2);
  glVertex2f(x1, y2);
  glEnd();
}
```

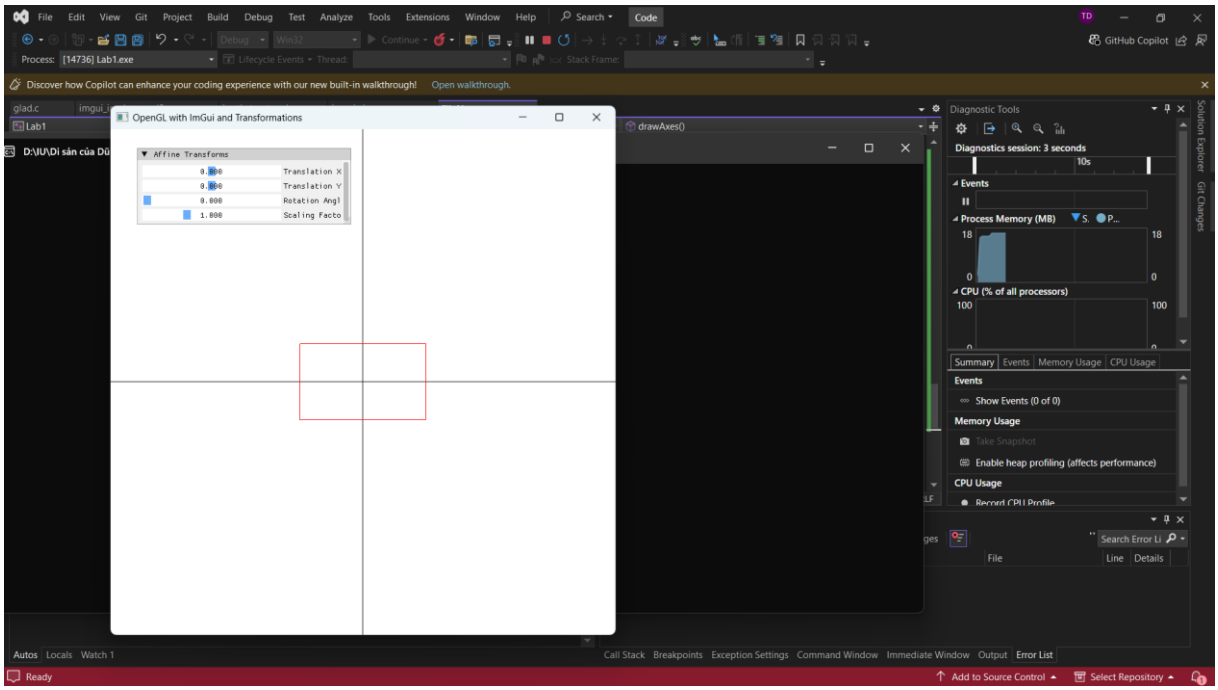
```
void drawLine(float x1, float y1, float x2, float y2, const ImVec4& color) {
    glColor3f(color.x, color.y, color.z);
    glBegin(GL_LINES);
    glVertex2f(x1, y1);
    glVertex2f(x2, y2);
    glEnd();
}

// Drawing a cubic Bézier curve
void drawCubicCurve(float x1, float y1, float x2, float y2, const ImVec4& color) {
    glColor3f(color.x, color.y, color.z);
    glBegin(GL_LINE_STRIP);
    for (float t = 0.0f; t <= 1.0f; t += 0.01f) {
        float xt = (1 - t) * (1 - t) * (1 - t) * x1
            + 3 * (1 - t) * (1 - t) * t * (x1 + (x2 - x1) / 4)
            + 3 * (1 - t) * t * t * (x2 - (x2 - x1) / 4)
            + t * t * t * x2;
        float yt = (1 - t) * (1 - t) * (1 - t) * y1
            + 3 * (1 - t) * (1 - t) * t * (y1 + (y2 - y1) / 2)
            + 3 * (1 - t) * t * t * (y2 - (y2 - y1) / 2)
            + t * t * t * y2;
        glVertex2f(xt, yt);
    }
    glEnd();
}

// Drawing a trapezoid (polygon)
void drawPolygon(float x1, float y1, float x2, float y2, const ImVec4& color) {
    glColor3f(color.x, color.y, color.z);
    glBegin(GL_LINE_LOOP);
    glVertex2f(x1 * 0.7f, y1);
    glVertex2f(x2 * 0.7f, y1);
    glVertex2f(x2, y2);
    glVertex2f(x1, y2);
    glEnd();
}

// Drawing a rounded rectangle
void drawRoundRectangle(float x1, float y1, float x2, float y2, const ImVec4& color) {
    glColor3f(color.x, color.y, color.z);
    float radius = 0.1f * (x2 - x1); // Corner radius is 10% of the rectangle width
    float width = x2 - x1;
    float height = y2 - y1;

    glBegin(GL_LINE_LOOP);

    // Top-right corner arc
    for (int i = 0; i <= 30; ++i) {
        float angle = i * 3.14159f / 60;
        float dx = cos(angle) * radius;
        float dy = sin(angle) * radius;
        glVertex2f(x2 - radius + dx, y2 - radius + dy);
    }

    // Bottom-right corner arc
    for (int i = 30; i <= 60; ++i) {
        float angle = i * 3.14159f / 60;
        float dx = cos(angle) * radius;
        float dy = sin(angle) * radius;
        glVertex2f(x1 + radius + dx, y2 - radius + dy);
    }

    // Bottom-left corner arc
    for (int i = 60; i <= 90; ++i) {
        float angle = i * 3.14159f / 60;
        float dx = cos(angle) * radius;
        float dy = sin(angle) * radius;
        glVertex2f(x1 + radius + dx, y1 + radius + dy);
    }

    // Top-left corner arc
    for (int i = 90; i <= 120; ++i) {
        float angle = i * 3.14159f / 60;
        float dx = cos(angle) * radius;
        float dy = sin(angle) * radius;
        glVertex2f(x2 - radius + dx, y1 + radius + dy);
    }

    glEnd();
}

// Drawing an ellipse
void drawEllipse(float x1, float y1, float x2, float y2, const ImVec4& color) {
    glColor3f(color.x, color.y, color.z);
    float centerX = (x1 + x2) / 2;
    float centerY = (y1 + y2) / 2;
    float radiusX = (x2 - x1) / 2;
    float radiusY = (y2 - y1) / 2;

    glBegin(GL_LINE_LOOP);
```

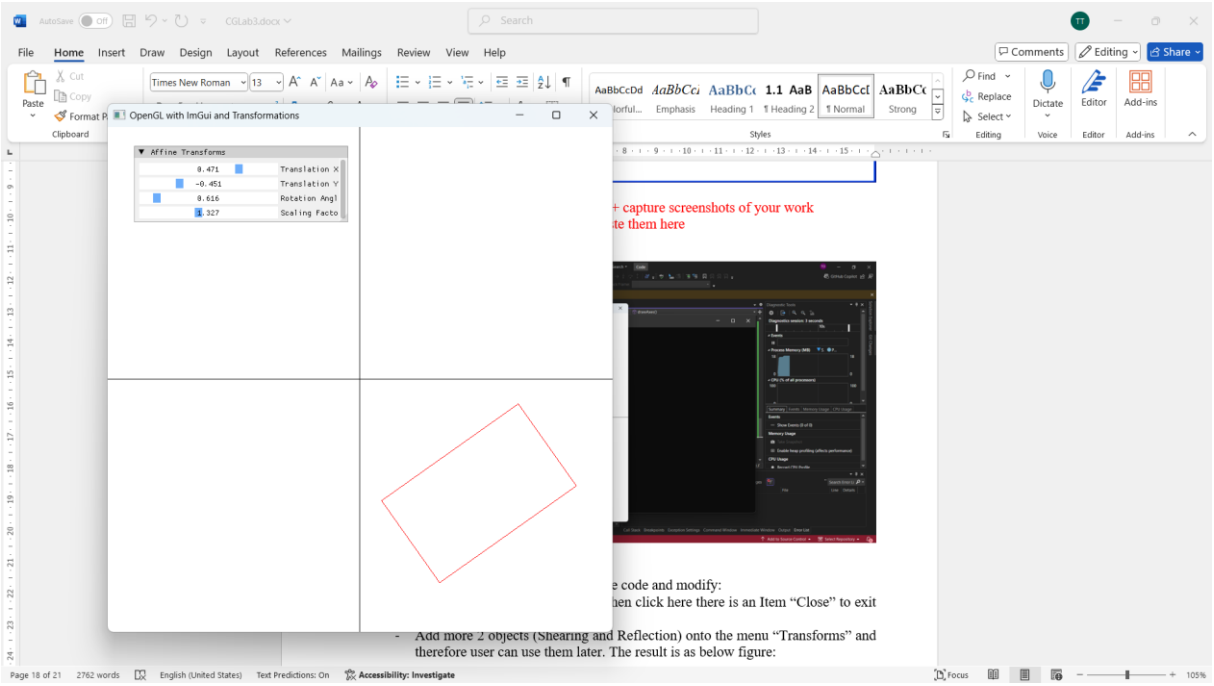
```
    for (int i = 0; i <= 360; ++i) {
        float theta = i * 3.14159f / 180.0f; // Convert degrees to radians
        float dx = cos(theta) * radiusX;
        float dy = sin(theta) * radiusY;
        glVertex2f(centerX + dx, centerY + dy);
    }
    glEnd();
}

// Drawing an arc
void drawArc(float x1, float y1, float x2, float y2, const ImVec4& color) {
    glColor3f(color.x, color.y, color.z);
    float centerX = (x1 + x2) / 2;
    float centerY = (y1 + y2) / 2;
    float radius = (x2 - x1) / 2;

    glBegin(GL_LINE_STRIP);
    for (int i = 45; i <= 135; ++i) {  // Drawing an arc from 45° to 135°
        float theta = i * 3.14159f / 180.0f; // Convert degrees to radians
        float dx = cos(theta) * radius;
        float dy = sin(theta) * radius;
        glVertex2f(centerX + dx, centerY + dy);
    }
    glEnd();
}

// Drawing a quadratic curve (using control points)
void drawQuadCurve(float x1, float y1, float x2, float y2, const ImVec4& color) {
    glColor3f(color.x, color.y, color.z);
    float controlX = (x1 + x2) / 2; // Control point for quadratic curve
    float controlY = y2 + 0.5f * (y2 - y1); // Control point to create the curve

    glBegin(GL_LINE_STRIP);
    for (float t = 0.0f; t <= 1.0f; t += 0.01f) {
        float x = (1 - t) * (1 - t) * x1 + 2 * (1 - t) * t * controlX + t * t * x2;
        float y = (1 - t) * (1 - t) * y1 + 2 * (1 - t) * t * controlY + t * t * y2;
        glVertex2f(x, y);
    }
    glEnd();
}
```

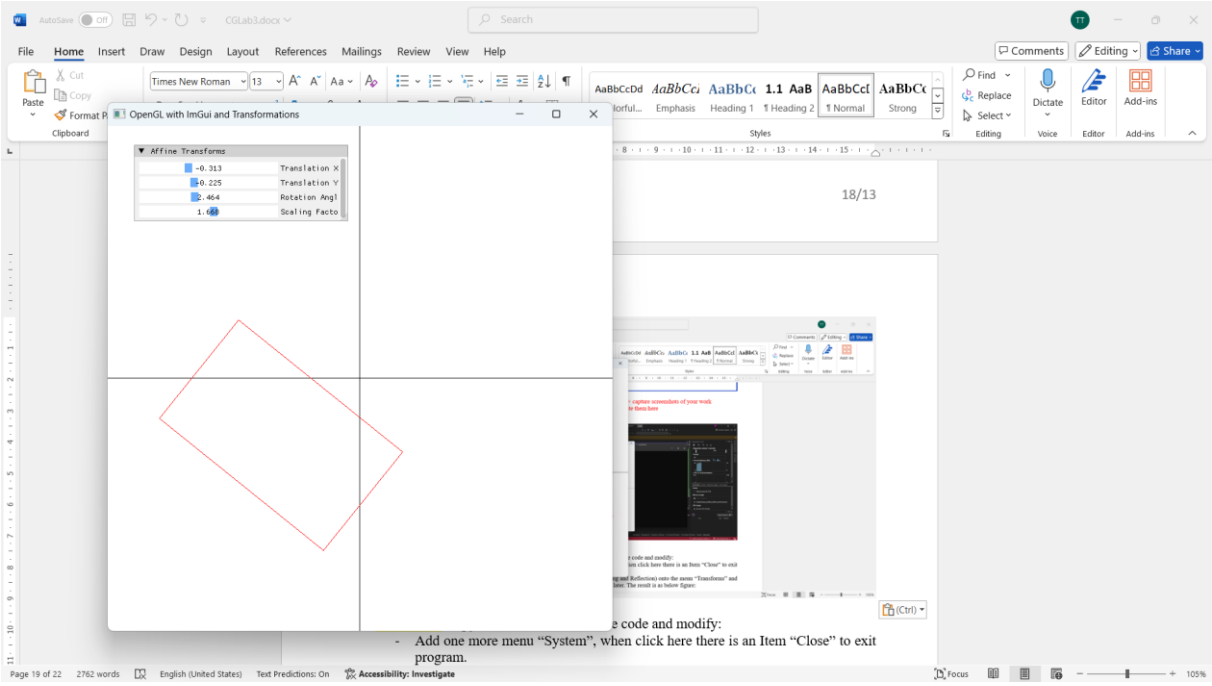
Part 2: Transformation

In this part, we will create a java application "Affine Transform". There is a menu "Transforms", when user click here, they can choose the objects in the dropdown menu to translate, rotate and scale.




OpenGL with ImGui and Transformations
Affine Transforms
Translation X
Translation Y
Rotation Angl
Scaling Facto
drawAxes()
Diagnostic Tools
Diagnostics session: 3 seconds
Events
Process Memory (MB)
CPU (% of all processors)
Summary
Memory Usage
CPU Usage
Show Events (0 of 0)
Enable heap profiling (affects performance)


OpenGL with ImGui and Transformations
Affine Transforms
0.471
-0.451
0.616
1.327
Translation X
Translation Y
Rotation Angl
Scaling Facto
+ capture screenshots of your work
te them here
e code and modify:
hen click here there is an Item "Close" to exit
Add more 2 objects (Shearing and Reflection) onto the menu "Transforms" and
therefore user can use them later. The result is as below figure:



This program demonstrates 2D affine transformations on a rectangle.

Global variables: translationX, translationY (float), rotationAngle (float, degrees), scalingFactor (float).

ImGui provides SliderFloat controls for each parameter.

The applyTransform() function applies transformations to each vertex (x, y) in order:

(1) Scale: x *= scalingFactor, y *= scalingFactor.

(2) Rotate: x' = x*cos(angle) - y*sin(angle), y' = x*sin(angle) + y*cos(angle), using <cmath> cos/sin.

(3) Translate: x += translationX, y += translationY. drawRectangle() applies the transform to 4 corner vertices then renders with GL_LINE_LOOP. drawAxes() draws x and y axes as reference lines using GL_LINES.

```
// Truong Tri Dung - MITIU25208
#define GLEW_STATIC
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include "imgui.h"
#include "imgui_impl_glfw.h"
#include "imgui_impl_opengl3.h"

#include <iostream>
#include <cmath> // For trigonometric functions
```

```
// Function declarations for drawing shapes
void drawRectangle(float x1, float y1, float x2, float y2, const ImVec4& color);
void drawAxes();
void applyTransform(float& x, float& y, float translationX, float translationY, float rotationAngle, float scalingFactor);

const int WINDOW_WIDTH = 800;
const int WINDOW_HEIGHT = 800;
```

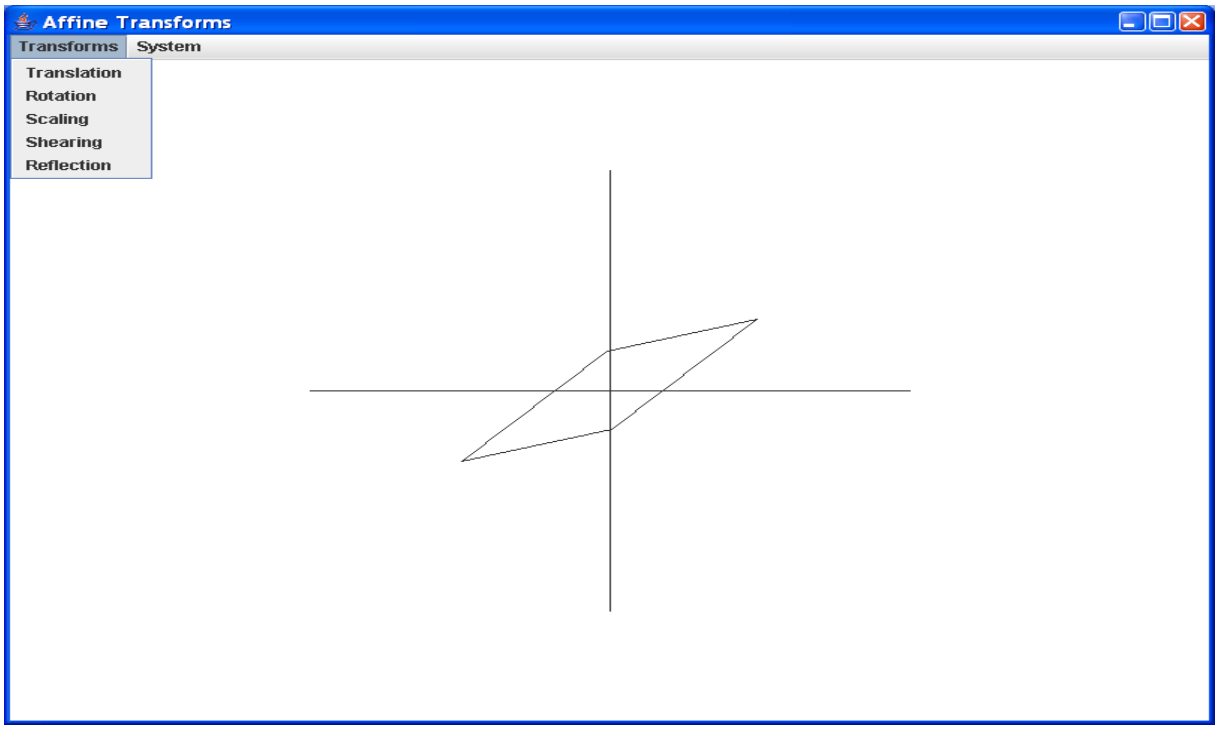
```
// Affine transformation variables
float translationX = 0.0f, translationY = 0.0f;
float rotationAngle = 0.0f;
float scalingFactor = 1.0f;

int main() {
    if (!glfwInit()) {
        std::cout << "Failed to initialize GLFW" << std::endl;
        return -1;
    }

    GLFWwindow* window = glfwCreateWindow(WINDOW_WIDTH, WINDOW_HEIGHT, "OpenGL with ImGui and Transformations", NULL, NULL);
    if (!window) {
        std::cout << "Failed to create GLFW window" << std::endl;
        glfwTerminate();
        return -1;
    }

    glfwMakeContextCurrent(window);
    glewExperimental = true;
    if (glewInit() != GLEW_OK) {
        std::cout << "Failed to initialize GLEW" << std::endl;
        return -1;
    }

    glViewport(0, 0, WINDOW_WIDTH, WINDOW_HEIGHT);

    // Initialize ImGui
    IMGUI_CHECKVERSION();
    ImGui::CreateContext();
    ImGuiIO& io = ImGui::GetIO(); (void)io;
    ImGui::StyleColorsLight();

ImGui_ImplGlfw_InitForOpenGL(window, true);
    ImGui_ImplOpenGL3_Init("#version 330");

    // Main loop
    while (!glfwWindowShouldClose(window)) {
        // Start the ImGui frame
        ImGui_ImplOpenGL3_NewFrame();
        ImGui_ImplGlfw_NewFrame();
        ImGui::NewFrame();

        // ImGui window for affine transformations
        ImGui::Begin("Affine Transforms");
        ImGui::SliderFloat("Translation X", &translationX, -1.0f, 1.0f);
        ImGui::SliderFloat("Translation Y", &translationY, -1.0f, 1.0f);
        ImGui::SliderFloat("Rotation Angle", &rotationAngle, 0.0f, 3.1415f * 2);
        ImGui::SliderFloat("Scaling Factor", &scalingFactor, 0.1f, 3.0f);
        ImGui::End();

        // Rendering OpenGL
        glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
        glClear(GL_COLOR_BUFFER_BIT);

        // Draw the axes
        drawAxes();

        // Draw a rectangle at (0, 0) with specified width and height
        float rectWidth = 0.5f;
```

```
        float rectHeight = 0.3f;
        drawRectangle(-rectWidth / 2, -rectHeight / 2, rectWidth / 2, rectHeight / 2, ImVec4(1.0f, 0.0f, 0.0f, 1.0f));

        // Render ImGui
        ImGui::Render();
ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());

        // Swap buffers and poll events
        glfwSwapBuffers(window);
        glfwPollEvents();
    }

    // Cleanup
    ImGui_ImplOpenGL3_Shutdown();
    ImGui_ImplGlfw_Shutdown();
    ImGui::DestroyContext();

    glfwDestroyWindow(window);
    glfwTerminate();

    return 0;
}

// Apply affine transformations manually
void applyTransform(float& x, float& y, float translationX, float translationY, float rotationAngle, float scalingFactor) {
    // Apply scaling
    x *= scalingFactor;
    y *= scalingFactor;

    // Apply rotation
    float rotatedX = x * cos(rotationAngle) - y * sin(rotationAngle);
    float rotatedY = x * sin(rotationAngle) + y * cos(rotationAngle);
    x = rotatedX;
    y = rotatedY;

    // Apply translation
    x += translationX;
    y += translationY;
}

// Drawing functions for shapes with transformation
void drawRectangle(float x1, float y1, float x2, float y2, const ImVec4& color) {
    glColor3f(color.x, color.y, color.z);
    glBegin(GL_LINE_LOOP);

    float savex1 = x1;
    float savex2 = x2;
    float savey1 = y1;
    float savey2 = y2;

    // Apply transformations to each vertex
    applyTransform(x1, y1, translationX, translationY, rotationAngle, scalingFactor);
    glVertex2f(x1, y1);

    x2 = savex2;
    y1 = savey1;
    applyTransform(x2, y1, translationX, translationY, rotationAngle, scalingFactor);
    glVertex2f(x2, y1);

    x2 = savex2;
    y2 = savey2;
    applyTransform(x2, y2, translationX, translationY, rotationAngle, scalingFactor);
    glVertex2f(x2, y2);
```

```
    x1 = savex1;
    y2 = savey2;
    applyTransform(x1, y2, translationX, translationY, rotationAngle, scalingFactor);
    glVertex2f(x1, y2);

    glEnd();
}

// Draw X and Y axes
void drawAxes() {
    // Set color for axes (black)
    glColor3f(0.0f, 0.0f, 0.0f);
    // Draw X axis (horizontal line from -1 to 1 at y=0)
    glBegin(GL_LINES);
    glVertex2f(-1.0f, 0.0f);
    glVertex2f(1.0f, 0.0f);
    glEnd();

    // Draw Y axis (vertical line from -1 to 1 at x=0)
    glBegin(GL_LINES);
    glVertex2f(0.0f, -1.0f);
    glVertex2f(0.0f, 1.0f);
    glEnd();
}
```

Exercise 2: Copy the source above source code and modify:

- Add one more menu "System", when click here there is an Item "Close" to exit program.
- Add more 2 objects (Shearing and Reflection) onto the menu "Transforms" and therefore user can use them later. The result is as below figure:

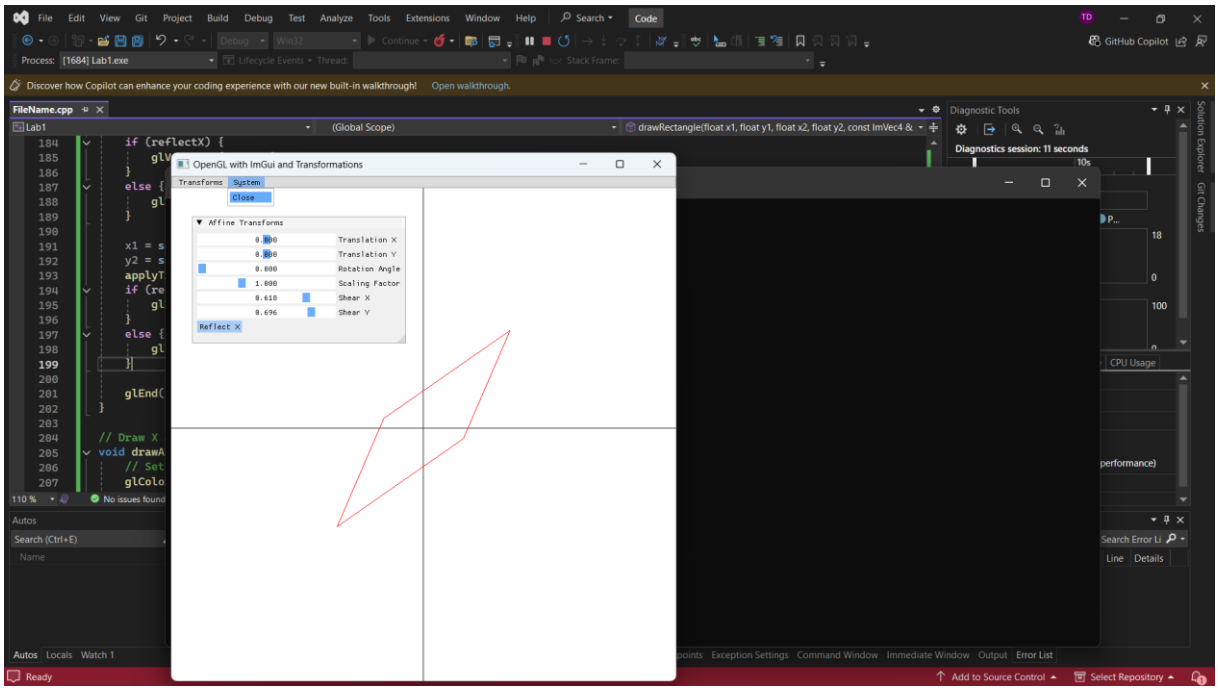


Hint: The case of Shearing:

double shx = ((double) (p1.x - x0) / (p.x - x0)) - 1;

```
double shy = ((double) (p1.y - y0) / (p.y - y0)) - 1;
tr.setToShear(shx, shy);
```

The case of Reflection: tr.setTransform(-1, 0, 0, 1, 0, 0); (you can try with the case of y reflection)



Extends the transformation program with shearX, shearY (float) and reflectX (bool).

ImGui adds SliderFloat for Shear X/Y and Checkbox for Reflect X.

The applyTransform() function is extended:

(1) After scaling, apply shearing: x' = x + shearX * y, y' = y + shearY * x. This implements the shear matrix [1, shx; shy, 1].

(2) Reflection: if reflectX is true, x = -x (flips across Y-axis), implementing the reflection matrix [-1, 0; 0, 1].

The menu bar uses ImGui::BeginMainMenuBar() with Transforms and System menus.

System > Close calls glfwSetWindowShouldClose(window, true).

```
// Truong Tri Dung - MITIU25208
#define GLEW_STATIC
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include "imgui.h"
#include "imgui_impl_glfw.h"
#include "imgui_impl_opengl3.h"
```

```
#include <iostream>
#include <cmath> // For trigonometric functions

// Function declarations for drawing shapes
void drawRectangle(float x1, float y1, float x2, float y2, const ImVec4& color);
void drawAxes();
```

```
void applyTransform(float& x, float& y, float translationX, float translationY, float rotationAngle, float scalingFactor, float shearX, float shearY);

// Window dimensions
const int WINDOW_WIDTH = 800;
const int WINDOW_HEIGHT = 800;

// Affine transformation variables
float translationX = 0.0f, translationY = 0.0f;
float rotationAngle = 0.0f;
float scalingFactor = 1.0f;
float shearX = 0.0f, shearY = 0.0f;
bool reflectX = false;

int main() {
    if (!glfwInit()) {
        std::cout << "Failed to initialize GLFW" << std::endl;
        return -1;
    }

    GLFWwindow* window = glfwCreateWindow(WINDOW_WIDTH, WINDOW_HEIGHT, "OpenGL with ImGui and Transformations", NULL, NULL);
    if (!window) {
        std::cout << "Failed to create GLFW window" << std::endl;
        glfwTerminate();
        return -1;
    }

    glfwMakeContextCurrent(window);
    glewExperimental = true;
    if (glewInit() != GLEW_OK) {
        std::cout << "Failed to initialize GLEW" << std::endl;
        return -1;
    }

    glViewport(0, 0, WINDOW_WIDTH, WINDOW_HEIGHT);

    // Initialize ImGui
    IMGUI_CHECKVERSION();
    ImGui::CreateContext();
    ImGuiIO& io = ImGui::GetIO(); (void)io;
    ImGui::StyleColorsLight();

ImGui_ImplGlfw_InitForOpenGL(window, true);
    ImGui_ImplOpenGL3_Init("#version 330");

    // Main loop
    while (!glfwWindowShouldClose(window)) {
        // Start the ImGui frame
        ImGui_ImplOpenGL3_NewFrame();
        ImGui_ImplGlfw_NewFrame();
        ImGui::NewFrame();

        // Create a menu bar
        if (ImGui::BeginMainMenuBar()) {
            if (ImGui::BeginMenu("Transforms")) {
                ImGui::MenuItem("Translation");
                ImGui::MenuItem("Rotation");
                ImGui::MenuItem("Scaling");
                ImGui::MenuItem("Shearing", NULL, &shearX);
                ImGui::MenuItem("Reflection", NULL, &reflectX);
                ImGui::EndMenu();
```

```
        }
        if (ImGui::BeginMenu("System")) {
            if (ImGui::MenuItem("Close")) {

glfwSetWindowShouldClose(window, GLFW_TRUE);
            }
            ImGui::EndMenu();
        }
        ImGui::EndMainMenuBar();
    }

    // ImGui window for affine transformations
    ImGui::Begin("Affine Transforms");
    ImGui::SliderFloat("Translation X", &translationX, -1.0f, 1.0f);
    ImGui::SliderFloat("Translation Y", &translationY, -1.0f, 1.0f);
    ImGui::SliderFloat("Rotation Angle", &rotationAngle, 0.0f, 3.1415f * 2);
    ImGui::SliderFloat("Scaling Factor", &scalingFactor, 0.1f, 3.0f);
    ImGui::SliderFloat("Shear X", &shearX, -5.0f, 5.0f);
    ImGui::SliderFloat("Shear Y", &shearY, -3.0f, 3.0f);

    // Button to reflect along X-axis
    if (ImGui::Button("Reflect X")) {
        reflectX = !reflectX;  // Toggle reflection
    }

    ImGui::End();

    // Rendering OpenGL
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    // Draw the axes
    drawAxes();

    // Draw a rectangle at (0, 0) with specified width and height
    float rectWidth = 0.5f;
    float rectHeight = 0.3f;
    drawRectangle(-rectWidth / 2, -rectHeight / 2, rectWidth / 2, rectHeight / 2, ImVec4(1.0f, 0.0f, 0.0f, 1.0f));

    // Render ImGui
    ImGui::Render();

ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());

    // Swap buffers and poll events
    glfwSwapBuffers(window);
    glfwPollEvents();
  }

  // Cleanup
  ImGui_ImplOpenGL3_Shutdown();
  ImGui_ImplGlfw_Shutdown();
  ImGui::DestroyContext();

  glfwDestroyWindow(window);
  glfwTerminate();

  return 0;
}

// Apply affine transformations manually
void applyTransform(float& x, float& y, float translationX, float translationY, float rotationAngle, float scalingFactor, float shearX, float shearY) {
```

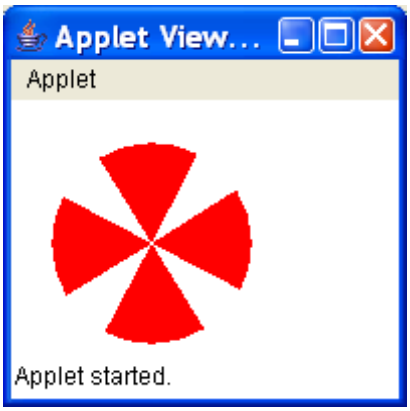
```
    // Shearing
    x += shearX * y;
    y += shearY * x;

    // Scaling
    x *= scalingFactor;
    y *= scalingFactor;

    // Rotation
    float rotatedX = x * cos(rotationAngle) - y * sin(rotationAngle);
    float rotatedY = x * sin(rotationAngle) + y * cos(rotationAngle);
    x = rotatedX;
    y = rotatedY;

    // Translation
    x += translationX;
    y += translationY;
}

// Drawing functions for shapes with transformation
void drawRectangle(float x1, float y1, float x2, float y2, const ImVec4& color) {
    glColor3f(color.x, color.y, color.z);
    glBegin(GL_LINE_LOOP);

    float savex1 = x1;
    float savex2 = x2;
    float savey1 = y1;
    float savey2 = y2;

    // Apply transformations to each vertex
    applyTransform(x1, y1, translationX, translationY, rotationAngle, scalingFactor, shearX, shearY);
    if (reflectX) {
        glVertex2f(-x1, y1);
    }
    else {
        glVertex2f(x1, y1);
    }

    x2 = savex2;
    y1 = savey1;
    applyTransform(x2, y1, translationX, translationY, rotationAngle, scalingFactor, shearX, shearY);
    if (reflectX) {
        glVertex2f(-x2, y1);
    }
    else {
        glVertex2f(x2, y1);
    }

    x2 = savex2;
    y2 = savey2;
    applyTransform(x2, y2, translationX, translationY, rotationAngle, scalingFactor, shearX, shearY);
    if (reflectX) {
        glVertex2f(-x2, y2);
    }
    else {
        glVertex2f(x2, y2);
    }

    x1 = savex1;
    y2 = savey2;
    applyTransform(x1, y2, translationX, translationY, rotationAngle, scalingFactor, shearX, shearY);
    if (reflectX) {
        glVertex2f(-x1, y2);
    }
```

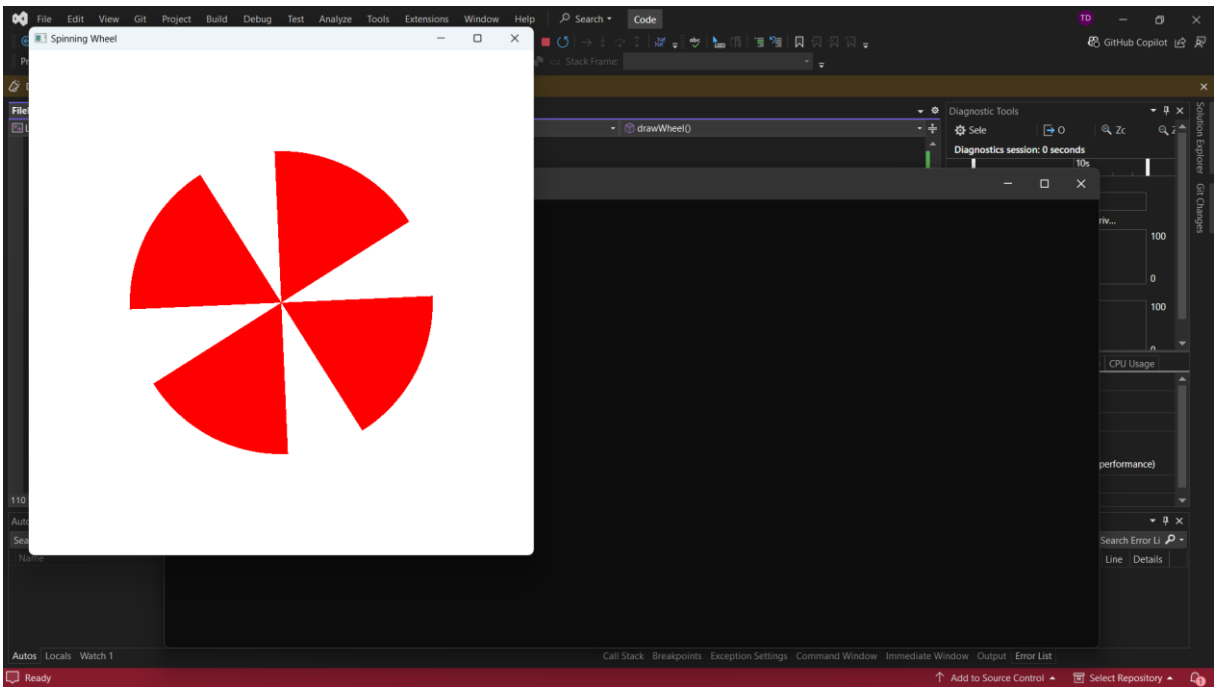
```
    else {
        glVertex2f(x1, y2);
    }

    glEnd();
}

// Draw X and Y axes
void drawAxes() {
    // Set color for axes (black)
    glColor3f(0.0f, 0.0f, 0.0f);

    // Draw X axis (horizontal line from -1 to 1 at y=0)
    glBegin(GL_LINES);
    glVertex2f(-1.0f, 0.0f);
    glVertex2f(1.0f, 0.0f);
    glEnd();

    // Draw Y axis (vertical line from -1 to 1 at x=0)
    glBegin(GL_LINES);
    glVertex2f(0.0f, -1.0f);
    glVertex2f(0.0f, 1.0f);
    glEnd();
}
```

Part 3: Animation

Draw a wheel:

The result:





Draws a spinning wheel with 8 blades using OpenGL immediate mode.

drawBlade(float angle): uses glPushMatrix/glPopMatrix with glRotatef(angle, 0, 0, 1) to orient each blade.

Each blade is a circular sector drawn with GL_TRIANGLE_FAN: center at (0,0), outer arc computed as x = 0.6*cos(theta), y = 0.6*sin(theta) for a 60-degree span (360/8 = 45 degrees between blades).

drawWheel() loops 8 times calling drawBlade(i * 45).

Animation: in the main loop, rotationAngle increments by 1.0f each frame, then glRotatef(rotationAngle, 0, 0, 1) is applied before drawWheel().

glfwSwapBuffers provides double buffering for smooth animation.

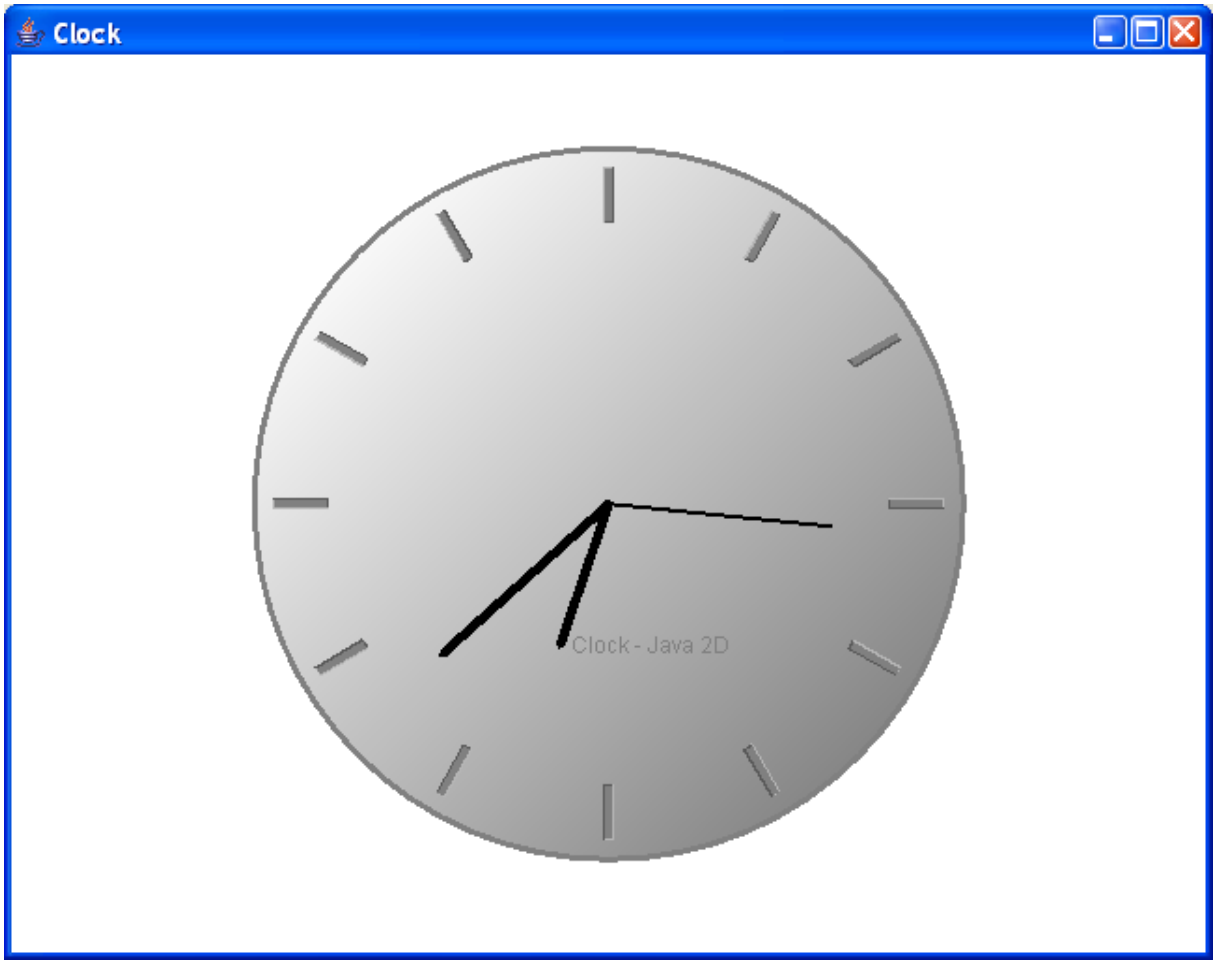
```
// Truong Tri Dung - MITIU25208
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <cmath> // for sin and cos

const int WINDOW_WIDTH = 800;
const int WINDOW_HEIGHT = 800;

float rotationAngle = 0.0f;  // Initial rotation angle

// Function to draw a fan blade (sector of a circle)
```

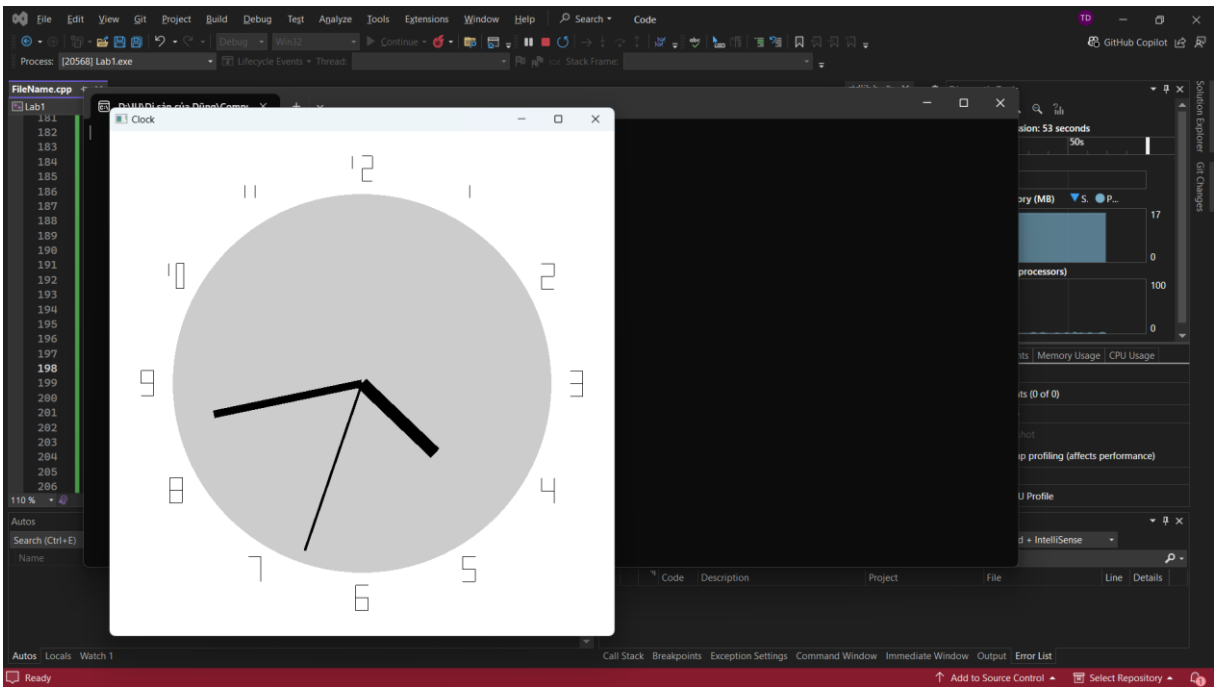
```
void drawBlade(float angle) {
    float radius = 0.6f;  // Radius of the circular sector
    int numSegments = 50;  // Number of segments to make the arc smooth
    float startAngle = angle * (3.1415f / 180.0f);  // Convert angle to radians
    float endAngle = startAngle + (60.0f * (3.1515f / 180.0f));  // 60 degrees for each sector

    glPushMatrix();
    glRotatef(angle, 0.0f, 0.0f, 1.0f);  // Rotate the blade
```

```
    glColor3f(1.0f, 0.0f, 0.0f);  // Red color for the blade

    // Draw the circular sector using GL_TRIANGLE_FAN
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(0.0f, 0.0f);  // Center of the wheel

    // Draw the outer edge of the sector
    for (int i = 0; i <= numSegments; ++i) {
        float theta = startAngle + i * (endAngle - startAngle) / numSegments;  // Angle of the current segment
        float x = radius * cos(theta);
        float y = radius * sin(theta);
        glVertex2f(x, y);  // Vertex on the arc
    }

    glEnd();
    glPopMatrix();
}


// Function to draw the wheel with multiple blades
void drawWheel() {
    int numBlades = 8;
    float angleStep = 360.0f / numBlades;

    for (int i = 0; i < numBlades; ++i) {
        drawBlade(i * angleStep);  // Draw each blade rotated
    }
}

int main() {
    if (!glfwInit()) {
        return -1;
    }

    GLFWwindow* window = glfwCreateWindow(WINDOW_WIDTH, WINDOW_HEIGHT, "Spinning Wheel", NULL, NULL);
    if (!window) {
        glfwTerminate();
        return -1;
    }

    glfwMakeContextCurrent(window);

    // Main loop
    while (!glfwWindowShouldClose(window)) {
        // Clear the window with white color
        glClear(GL_COLOR_BUFFER_BIT);
        glClearColor(1.0f, 1.0f, 1.0f, 1.0f);

        // Apply rotation
        glPushMatrix();
        glTranslatef(0.0f, 0.0f, 0.0f);  // Move to the center of the screen
        glRotatef(rotationAngle, 0.0f, 0.0f, 1.0f);  // Rotate the whole wheel

        // Draw the wheel
        drawWheel();

        glPopMatrix();

        // Increment the rotation angle for animation
        rotationAngle += 0.05f;
        if (rotationAngle > 360.0f) {
```

```
        rotationAngle -= 360.0f;
    }

    // Swap buffers and poll events
    glfwSwapBuffers(window);
    glfwPollEvents();
    }

    glfwTerminate();
    return 0;
}
```

Exercise 3: **Draw a Clock as below but change all "|" = number (from 1 to 12)**





Draws an analog clock with numbers 1-12 and animated hands.

drawClockFace(): uses GL_TRIANGLE_FAN with 100 segments, radius=0.75, light gray fill.

drawDigit(int digit, float x, float y, float scale): renders digits 1-12 using GL_LINES with a switch-case defining line segments for each number (e.g., case 2: five line segments for the shape of '2').

Numbers are positioned using: x = 0.6*sin(i*30 degrees), y = 0.6*cos(i*30 degrees) for i=1..12.

drawHand(float angle, float length, float r, float g, float b): draws a line from center (0,0) to (length*sin(angle), length*cos(angle)).

Time is obtained via glfwGetTime(); secondAngle = fmod(time, 60) * 6 degrees, minuteAngle = fmod(time/60, 60) * 6 degrees, hourAngle = fmod(time/3600, 12) * 30 degrees.

```
// Truong Tri Dung - MITIU25208
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <cmath>

const int WINDOW_WIDTH = 800;
const int WINDOW_HEIGHT = 800;

float hourAngle = 0.0f;
float minuteAngle = 0.0f;
float secondAngle = 0.0f;

// Function to draw the clock face (a circle)
void drawClockFace() {
    float radius = 0.75f;  // Radius of the clock face
    int numSegments = 100;  // More segments for a smoother circle

    glColor3f(0.8f, 0.8f, 0.8f);  // Light gray for clock face
```

```
    // Draw the circle
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(0.0f, 0.0f);  // Center of the clock
    for (int i = 0; i <= numSegments; ++i) {
        float theta = 2.0f * 3.1415f * i / numSegments;
        float x = radius * cos(theta);
        float y = radius * sin(theta);
        glVertex2f(x, y);
    }
    glEnd();
}

// Function to draw a digit (supporting 1 to 12)
void drawDigit(int digit, float x, float y, float scale) {
    glColor3f(0.0f, 0.0f, 0.0f);  // Black color for the numbers

    glPushMatrix();
    glTranslatef(x, y, 0.0f);  // Move to position
    glScalef(scale, scale, 1.0f);  // Scale the digit size

    glBegin(GL_LINES);

    // Define line segments for digits 1 to 12
    switch (digit) {
    case 1:  // Number 1
        glVertex2f(0.0f, 0.0f); glVertex2f(0.0f, 0.5f);
        break;
    case 2:  // Number 2
        glVertex2f(-0.25f, 0.5f); glVertex2f(0.25f, 0.5f);
        glVertex2f(0.25f, 0.5f); glVertex2f(0.25f, 0.0f);
        glVertex2f(0.25f, 0.0f); glVertex2f(-0.25f, 0.0f);
        glVertex2f(-0.25f, 0.0f); glVertex2f(-0.25f, -0.5f);
        glVertex2f(-0.25f, -0.5f); glVertex2f(0.25f, -0.5f);
        break;
    case 3:  // Number 3
        glVertex2f(-0.25f, 0.5f); glVertex2f(0.25f, 0.5f);
        glVertex2f(0.25f, 0.5f); glVertex2f(0.25f, 0.0f);
        glVertex2f(-0.25f, 0.0f); glVertex2f(0.25f, 0.0f);
        glVertex2f(0.25f, 0.0f); glVertex2f(0.25f, -0.5f);
        glVertex2f(-0.25f, -0.5f); glVertex2f(0.25f, -0.5f);
        break;
    case 4:  // Number 4
        glVertex2f(-0.25f, 0.5f); glVertex2f(-0.25f, 0.0f);
        glVertex2f(-0.25f, 0.0f); glVertex2f(0.25f, 0.0f);
        glVertex2f(0.25f, 0.5f); glVertex2f(0.25f, -0.5f);
        break;
    case 5:  // Number 5
        glVertex2f(0.25f, 0.5f); glVertex2f(-0.25f, 0.5f);
        glVertex2f(-0.25f, 0.5f); glVertex2f(-0.25f, 0.0f);
        glVertex2f(-0.25f, 0.0f); glVertex2f(0.25f, 0.0f);
        glVertex2f(0.25f, 0.0f); glVertex2f(0.25f, -0.5f);
        glVertex2f(-0.25f, -0.5f); glVertex2f(0.25f, -0.5f);
        break;
```

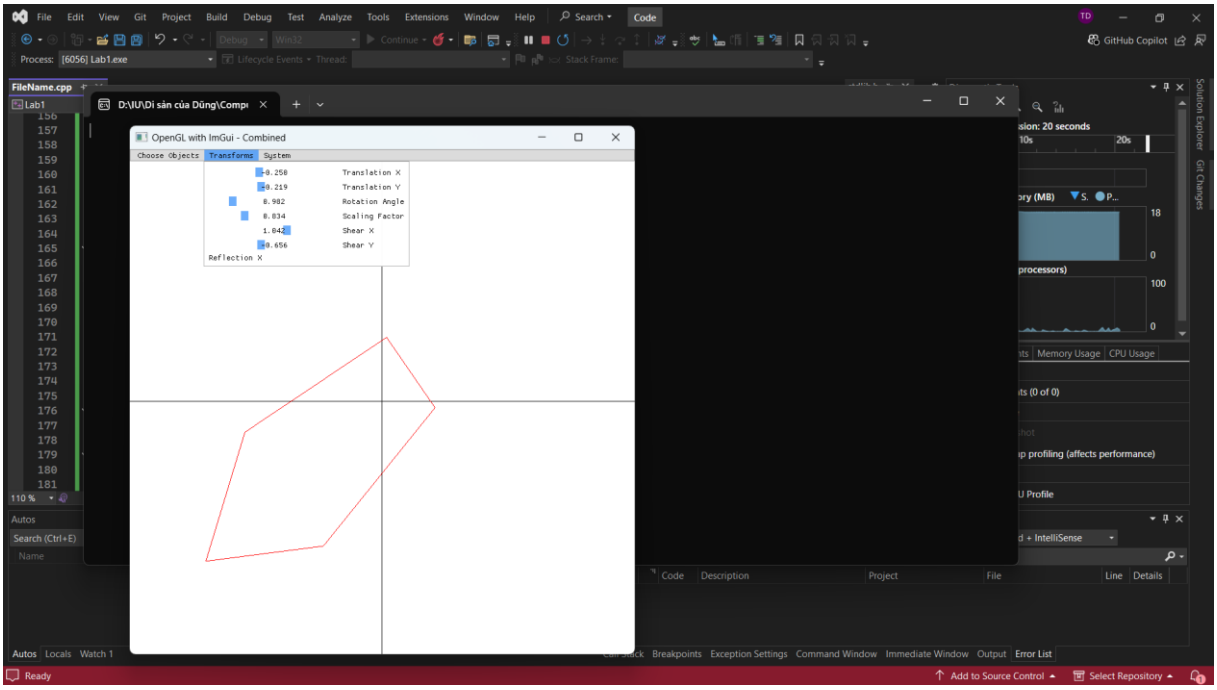
```
    case 6:  // Number 6
        glVertex2f(0.25f, 0.5f); glVertex2f(-0.25f, 0.5f);
        glVertex2f(-0.25f, 0.5f); glVertex2f(-0.25f, -0.5f);
        glVertex2f(-0.25f, 0.0f); glVertex2f(0.25f, 0.0f);
        glVertex2f(0.25f, 0.0f); glVertex2f(0.25f, -0.5f);
        glVertex2f(-0.25f, -0.5f); glVertex2f(0.25f, -0.5f);
        break;
    case 7:  // Number 7
        glVertex2f(-0.25f, 0.5f); glVertex2f(0.25f, 0.5f);
        glVertex2f(0.25f, 0.5f); glVertex2f(0.25f, -0.5f);
        break;
    case 8:  // Number 8
        glVertex2f(-0.25f, 0.5f); glVertex2f(0.25f, 0.5f);
        glVertex2f(-0.25f, 0.0f); glVertex2f(0.25f, 0.0f);
        glVertex2f(-0.25f, -0.5f); glVertex2f(0.25f, -0.5f);
        glVertex2f(-0.25f, 0.5f); glVertex2f(-0.25f, -0.5f);
        glVertex2f(0.25f, 0.5f); glVertex2f(0.25f, -0.5f);
        break;
    case 9:  // Number 9
        glVertex2f(-0.25f, -0.5f); glVertex2f(0.25f, -0.5f);
        glVertex2f(0.25f, 0.5f); glVertex2f(0.25f, -0.5f);
        glVertex2f(-0.25f, 0.5f); glVertex2f(0.25f, 0.5f);
        glVertex2f(-0.25f, 0.5f); glVertex2f(-0.25f, 0.0f);
        glVertex2f(-0.25f, 0.0f); glVertex2f(0.25f, 0.0f);
        break;
    case 10: // Number 10
        // Draw '1'
        glVertex2f(-0.3f, 0.0f); glVertex2f(-0.3f, 0.5f);
        // Draw '0'
        glVertex2f(0.0f, 0.5f); glVertex2f(0.0f, -0.5f);
        glVertex2f(0.0f, -0.5f); glVertex2f(0.3f, -0.5f);
        glVertex2f(0.3f, -0.5f); glVertex2f(0.3f, 0.5f);
        glVertex2f(0.3f, 0.5f); glVertex2f(0.0f, 0.5f);
        break;
    case 11: // Number 11
        // Draw first '1'
        glVertex2f(-0.4f, 0.0f); glVertex2f(-0.4f, 0.5f);
        // Draw second '1'
        glVertex2f(0.0f, 0.0f); glVertex2f(0.0f, 0.5f);
        break;
    case 12: // Number 12
        // Draw '1'
        glVertex2f(-0.4f, 0.0f); glVertex2f(-0.4f, 0.5f);
        // Draw '2'
        glVertex2f(0.0f, 0.5f); glVertex2f(0.4f, 0.5f);
        glVertex2f(0.4f, 0.5f); glVertex2f(0.4f, 0.0f);
        glVertex2f(0.4f, 0.0f); glVertex2f(0.0f, 0.0f);
        glVertex2f(0.0f, 0.0f); glVertex2f(0.0f, -0.5f);
        glVertex2f(0.0f, -0.5f); glVertex2f(0.4f, -0.5f);
        break;
    }
```

```
    glEnd();
    glPopMatrix();
}

// Function to draw hour numbers with swapped positions
void drawHourNumbers() {
    float radius = 0.85f;  // Radius where the numbers are drawn
    float scale = 0.1f;    // Scale of the numbers
    int numbers[12] = { 2, 1, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3 };  // Swapped number order

    for (int i = 0; i < 12; ++i) {
        float angle = (i + 1) * (360.0f / 12);  // Calculate angle for each number
        float rad = angle * (3.1415f / 180.0f); // Convert to radians

        // Calculate x and y positions for each number based on angle and radius
        float x = radius * cos(rad);
        float y = radius * sin(rad);

        // Draw the number at the calculated position
        drawDigit(numbers[i], x, y, scale);
    }
}

// Function to draw clock hands
void drawHand(float angle, float length, float width) {
    glPushMatrix();
    glRotatef(angle, 0.0f, 0.0f, 1.0f);  // Rotate to the given angle
    glColor3f(0.0f, 0.0f, 0.0f);  // Black color for clock hands
    glBegin(GL_QUADS);  // Draw hand as a rectangle
    glVertex2f(-width / 2, 0.0f);
    glVertex2f(width / 2, 0.0f);
    glVertex2f(width / 2, length);
    glVertex2f(-width / 2, length);
    glEnd();

    glPopMatrix();
}

int main() {
    if (!glfwInit()) {
        return -1;
    }

    GLFWwindow* window = glfwCreateWindow(WINDOW_WIDTH, WINDOW_HEIGHT, "Clock", NULL, NULL);
    if (!window) {
        glfwTerminate();
        return -1;
    }

    glfwMakeContextCurrent(window);

    // Main loop
    while (!glfwWindowShouldClose(window)) {
        // Clear the window with white color
        glClear(GL_COLOR_BUFFER_BIT);
        glClearColor(1.0f, 1.0f, 1.0f, 1.0f);

        // Draw the clock face
        drawClockFace();
```

```
        // Draw the hour numbers
        drawHourNumbers();  // Replace hour ticks with numbers

        // Draw the clock hands
        drawHand(hourAngle, 0.4f, 0.05f);   // Hour hand
        drawHand(minuteAngle, 0.6f, 0.03f); // Minute hand
        drawHand(secondAngle, 0.7f, 0.01f); // Second hand

        // Increment angles for simple animation
        secondAngle -= 0.05f;
        minuteAngle -= 0.01f;
        hourAngle -= 0.001f;

        // Swap buffers and poll events
        glfwSwapBuffers(window);
        glfwPollEvents();
    }

    glfwTerminate();
    return 0;
}
```

Homework:

1 - Combine ex1 and ex2 in order to have a full program with 3 menus on the menu bar: (Choose Objects, Transforms, System)



Merges shape drawing (Exercise 1) and transformations (Exercise 2) into one application. Defines ObjectType enum {RECTANGLE, POLYGON} and currentObject for object selection.

drawPolygon(int numSides, float radius, ImVec4 color): draws a regular polygon using GL_LINE_LOOP with vertices at radius*cos(i*2*PI/numSides), radius*sin(i*2*PI/numSides).

ImGui menu bar has 3 menus: Choose Objects (Rectangle/Polygon with numPolygonSides slider), Transforms (SliderFloat for Translation X/Y, Rotation, Scaling, Shear X/Y, and Checkbox for Reflect X), System (Close).

applyTransform() applies scale -> shear -> rotate -> translate -> reflect in sequence to each vertex.

The selected object is drawn with the current transformation applied.

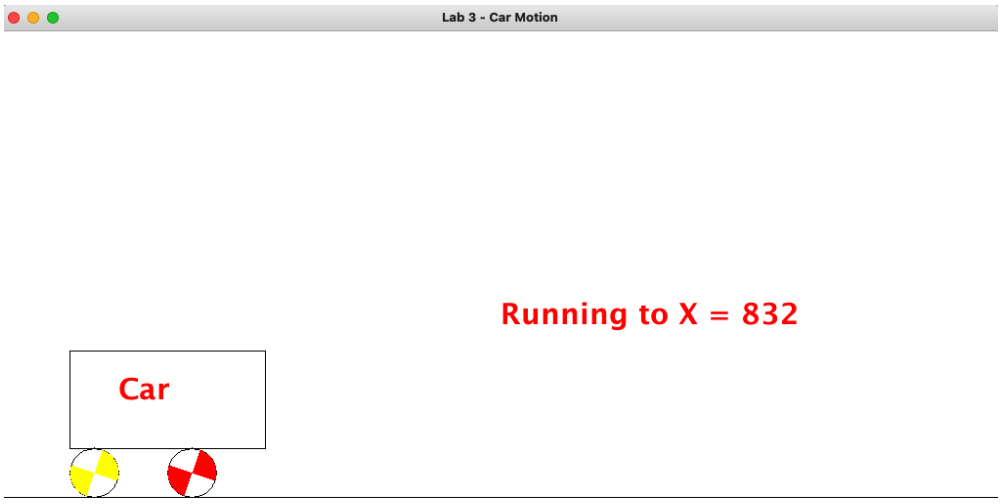
```
// Truong Tri Dung - MITIU25208
#define GLEW_STATIC
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include "imgui.h"
#include "imgui_impl_glfw.h"
#include "imgui_impl_opengl3.h"
#include <iostream>
#include <cmath> // For trigonometric functions

// Function declarations for drawing shapes
void drawRectangle(float x1, float y1, float x2, float y2, const ImVec4& color);
void drawPolygon(int numSides, float radius, const ImVec4& color);
void drawAxes();
void applyTransform(float& x, float& y, float translationX, float translationY, float rotationAngle, float scalingFactor, float shearX, float shearY);

// Window dimensions
const int WINDOW_WIDTH = 800;
const int WINDOW_HEIGHT = 800;

// Object selection enum
enum ObjectType { RECTANGLE, POLYGON };
ObjectType currentObject = RECTANGLE; // Default object

// Polygon parameters
int numPolygonSides = 5;

// Affine transformation variables
float translationX = 0.0f, translationY = 0.0f;
float rotationAngle = 0.0f;
float scalingFactor = 1.0f;
float shearX = 0.0f, shearY = 0.0f;
bool reflectX = false;

int main() {
    if (!glfwInit()) {
        std::cout << "Failed to initialize GLFW" << std::endl;
        return -1;
    }

    GLFWwindow* window = glfwCreateWindow(WINDOW_WIDTH, WINDOW_HEIGHT, "OpenGL with ImGui - Combined", NULL, NULL);
    if (!window) {
        std::cout << "Failed to create GLFW window" << std::endl;
        glfwTerminate();
        return -1;
    }

    glfwMakeContextCurrent(window);
    glewExperimental = true;
    if (glewInit() != GLEW_OK) {
        std::cout << "Failed to initialize GLEW" << std::endl;
        return -1;
    }

    glViewport(0, 0, WINDOW_WIDTH, WINDOW_HEIGHT);

    // Initialize ImGui
    IMGUI_CHECKVERSION();
    ImGui::CreateContext();
    ImGuiIO& io = ImGui::GetIO(); (void)io;
    ImGui::StyleColorsLight();
```

```
ImGui_ImplGlfw_InitForOpenGL(window, true);
   ImGui_ImplOpenGL3_Init("#version 330");

   // Main loop
   while (!glfwWindowShouldClose(window)) {
      // Start the ImGui frame
      ImGui_ImplOpenGL3_NewFrame();
      ImGui_ImplGlfw_NewFrame();
      ImGui::NewFrame();

      // Create a menu bar
      if (ImGui::BeginMainMenuBar()) {
         // Choose Objects Menu
         if (ImGui::BeginMenu("Choose Objects")) {
            if (ImGui::MenuItem("Rectangle", NULL, currentObject == RECTANGLE)) {
               currentObject = RECTANGLE;
            }
            if (ImGui::MenuItem("Polygon", NULL, currentObject == POLYGON)) {
               currentObject = POLYGON;
            }
            ImGui::EndMenu();
         }

         // Transforms Menu
         if (ImGui::BeginMenu("Transforms")) {
            ImGui::SliderFloat("Translation X", &translationX, -1.0f, 1.0f);
            ImGui::SliderFloat("Translation Y", &translationY, -1.0f, 1.0f);
            ImGui::SliderFloat("Rotation Angle", &rotationAngle, 0.0f, 3.1415f * 2);
            ImGui::SliderFloat("Scaling Factor", &scalingFactor, 0.1f, 3.0f);
            ImGui::SliderFloat("Shear X", &shearX, -5.0f, 5.0f);
            ImGui::SliderFloat("Shear Y", &shearY, -3.0f, 3.0f);
            ImGui::MenuItem("Reflection X", NULL, &reflectX);
            ImGui::EndMenu();
         }

         // System Menu
         if (ImGui::BeginMenu("System")) {
            if (ImGui::MenuItem("Close")) {
               glfwSetWindowShouldClose(window, GLFW_TRUE);
            }
            ImGui::EndMenu();
         }

         ImGui::EndMainMenuBar();
      }

      // Rendering OpenGL
      glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
      glClear(GL_COLOR_BUFFER_BIT);

      // Draw the axes
      drawAxes();

      // Draw selected object
      ImVec4 color = ImVec4(1.0f, 0.0f, 0.0f, 1.0f); // Red color
      if (currentObject == RECTANGLE) {
         drawRectangle(-0.5f, -0.3f, 0.5f, 0.3f, color);
      }
      else if (currentObject == POLYGON) {
         drawPolygon(numPolygonSides, 0.5f, color);
      }

      // Render ImGui
      ImGui::Render();
      ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());

      // Swap buffers and poll events
      glfwSwapBuffers(window);
      glfwPollEvents();
   }

   // Cleanup
   ImGui_ImplOpenGL3_Shutdown();
   ImGui_ImplGlfw_Shutdown();
   ImGui::DestroyContext();
   glfwDestroyWindow(window);
   glfwTerminate();

   return 0;
}
```

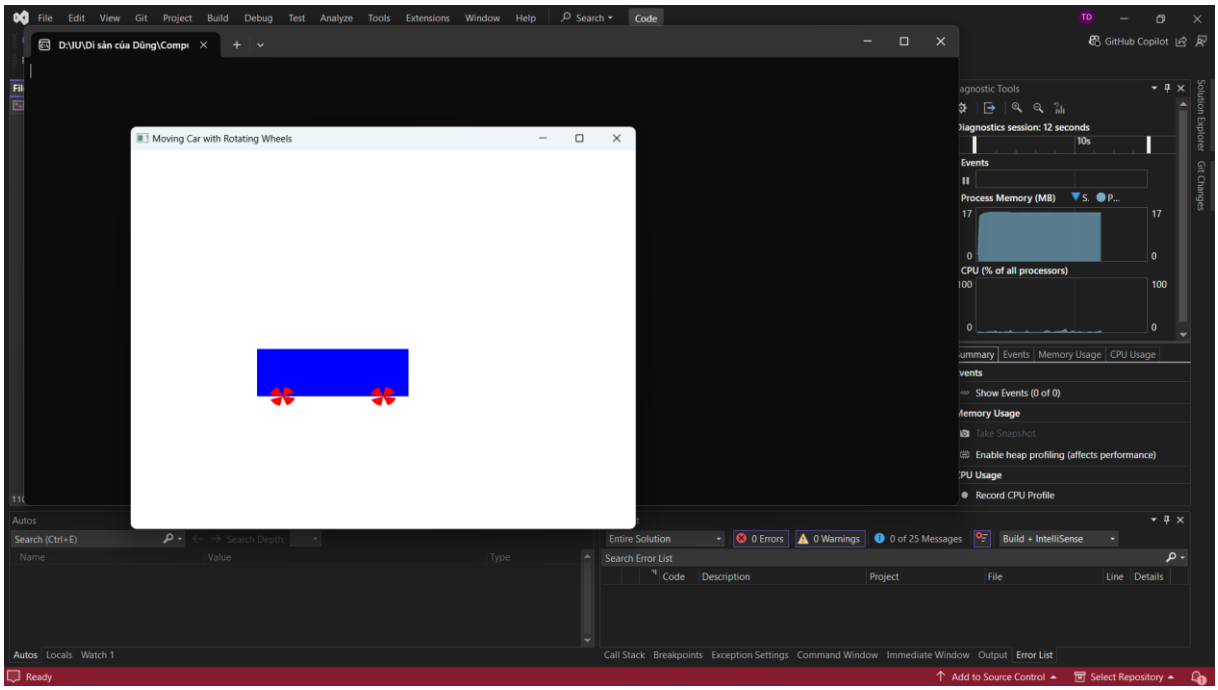
```
// Apply affine transformations manually
void applyTransform(float& x, float& y,
float translationX, float translationY, float
rotationAngle, float scalingFactor, float
shearX, float shearY) {
   // Shearing
   x += shearX * y;
   y += shearY * x;

   // Scaling
   x *= scalingFactor;
   y *= scalingFactor;

   // Rotation
   float rotatedX = x * cos(rotationAngle) -
y * sin(rotationAngle);
   float rotatedY = x * sin(rotationAngle) +
y * cos(rotationAngle);
   x = rotatedX;
   y = rotatedY;

   // Translation
   x += translationX;
   y += translationY;
}

// Drawing functions for shapes with
transformation
void drawRectangle(float x1, float y1, float
x2, float y2, const ImVec4& color) {
   glColor3f(color.x, color.y, color.z);
   glBegin(GL_LINE_LOOP);

   float savex1 = x1;
   float savex2 = x2;
   float savey1 = y1;
   float savey2 = y2;

   // Apply transformations to each vertex
   applyTransform(x1, y1, translationX,
translationY, rotationAngle, scalingFactor,
shearX, shearY);
   if (reflectX) {
      glVertex2f(-x1, y1);
   }
   else {
      glVertex2f(x1, y1);
   }

   x2 = savex2;
   y1 = savey1;
   applyTransform(x2, y1, translationX,
translationY, rotationAngle, scalingFactor,
shearX, shearY);
   if (reflectX) {
      glVertex2f(-x2, y1);
   }
   else {
      glVertex2f(x2, y1);
   }

   x2 = savex2;
   y2 = savey2;
   applyTransform(x2, y2, translationX,
translationY, rotationAngle, scalingFactor,
shearX, shearY);
   if (reflectX) {
      glVertex2f(-x2, y2);
   }
   else {
      glVertex2f(x2, y2);
   }

   x1 = savex1;
   y2 = savey2;
   applyTransform(x1, y2, translationX,
translationY, rotationAngle, scalingFactor,
shearX, shearY);
   if (reflectX) {
      glVertex2f(-x1, y2);
   }
   else {
      glVertex2f(x1, y2);
   }

   glEnd();
}

// Draw a polygon with specified number of
sides
void drawPolygon(int numSides, float
radius, const ImVec4& color) {
   glColor3f(color.x, color.y, color.z);
   glBegin(GL_LINE_LOOP);
   for (int i = 0; i < numSides; i++) {
      float angle = i * 2.0f * 3.1415f /
numSides;
      float x = radius * cos(angle);
      float y = radius * sin(angle);
      applyTransform(x, y, translationX,
translationY, rotationAngle, scalingFactor,
shearX, shearY);
      if (reflectX) x = -x;
```

```
        glVertex2f(x, y);
    }
    glEnd();
}

// Draw X and Y axes
void drawAxes() {
    glColor3f(0.0f, 0.0f, 0.0f);
    glBegin(GL_LINES);
    glVertex2f(-1.0f, 0.0f);
    glVertex2f(1.0f, 0.0f);
    glEnd();
    glBegin(GL_LINES);
    glVertex2f(0.0f, -1.0f);
    glVertex2f(0.0f, 1.0f);
    glEnd();
}
```

2 - Draw a car (Draw a wheel of a car, Rotate by timer, Move along x-axis)



Left mouse click to run the car!



Draws a moving car with rotating wheels using OpenGL.

Global state: carPosX (float, starts at -1.0), wheelAngle (float), isMousePressed (bool).

mouseButtonCallback: sets isMousePressed on left-click to control animation.

drawWheel(float cx, float cy, float radius): draws a circle with GL_LINE_LOOP (100 segments) and 8 spokes using GL_LINES, rotated by wheelAngle using glRotatef.

drawCar(): draws car body with GL_QUADS (chassis rectangle) and GL_TRIANGLES (roof), then calls drawWheel() at two positions for front and rear wheels.

The entire car is translated by glTranslatef(carPosX, 0, 0).

Animation: when isMousePressed, carPosX += 0.005f and wheelAngle += 2.0f each frame.

When carPosX > 1.5, it resets to -1.5 for looping.

```
// Truong Tri Dung - MITIU25208
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <cmath>  // for sin, cos

const int WINDOW_WIDTH = 800;
const int WINDOW_HEIGHT = 600;

float carPosX = -1.0f;   // Initial x position of the car
float wheelAngle = 0.0f; // Wheel rotation angle
bool isMousePressed = false; // Flag to check if the mouse is pressed

// Function to handle mouse button events
void mouseButtonCallback(GLFWwindow* window, int button, int action, int mods) {
    if (button == GLFW_MOUSE_BUTTON_LEFT) {
        if (action == GLFW_PRESS) {
            isMousePressed = true;  // Mouse is pressed
        } else if (action == GLFW_RELEASE) {
            isMousePressed = false; // Mouse is released
        }
    }
}

// Function to draw a fan blade (sector of a circle)
void drawBlade(float angle) {
    float radius = 0.05f;  // Radius of the circular sector
    int numSegments = 50;  // Number of segments to make the arc smooth
    float startAngle = angle * (3.1415f / 180.0f);  // Convert angle to radians
    float endAngle = startAngle + (60.0f * (3.1515f / 180.0f));  // 60 degrees for each sector

    glPushMatrix();
    glRotatef(angle, 0.0f, 0.0f, 1.0f);  // Rotate the blade

    glColor3f(1.0f, 0.0f, 0.0f);  // Red color for the blade

    // Draw the circular sector using GL_TRIANGLE_FAN
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(0.0f, 0.0f);  // Center of the wheel

    // Draw the outer edge of the sector
    for (int i = 0; i <= numSegments; ++i) {
        float theta = startAngle + i * (endAngle - startAngle) / numSegments;  // Angle of the current segment
        float x = radius * cos(theta);
        float y = radius * sin(theta);
        glVertex2f(x, y);  // Vertex on the arc
    }
```

```
    glEnd();
    glPopMatrix();
}

// Function to draw a wheel
void drawWheel(float x, float y, float
radius) {
    int numBlades = 8;
    float angleStep = 360.0f / numBlades;

    for (int i = 0; i < numBlades; ++i) {
        drawBlade(i * angleStep);  // Draw
each blade rotated
    }
}

// Function to draw the car body
void drawCarBody(float x, float y) {
    glColor3f(0.0f, 0.0f, 1.0f);  // Blue color
for the car body
    glBegin(GL_QUADS);
    glVertex2f(x - 0.3f, y - 0.1f);  // Bottom-
left corner
    glVertex2f(x + 0.3f, y - 0.1f);  // Bottom-
right corner
    glVertex2f(x + 0.3f, y + 0.15f);  // Top-
right corner
    glVertex2f(x - 0.3f, y + 0.15f);  // Top-left
corner
    glEnd();
}

// Function to draw the car (body + wheels)
void drawCar(float x) {
    float wheelRadius = 0.05f;
    float wheelOffsetX = 0.2f;
    float wheelOffsetY = -0.12f;

    // Draw the car body
    drawCarBody(x, -0.2f);

    // Draw the two wheels with rotation
    glPushMatrix();
    glTranslatef(x - wheelOffsetX, -0.3f,
0.0f);  // Position the left wheel
    glRotatef(wheelAngle, 0.0f, 0.0f, 1.0f);  //
Rotate the wheel
    drawWheel(0.0f, 0.0f, wheelRadius);
    glPopMatrix();

    glPushMatrix();
    glTranslatef(x + wheelOffsetX, -0.3f,
0.0f);  // Position the right wheel
    glRotatef(wheelAngle, 0.0f, 0.0f, 1.0f);  //
Rotate the wheel
    drawWheel(0.0f, 0.0f, wheelRadius);
    glPopMatrix();
}

int main() {
    if (!glfwInit()) {
        return -1;
    }

    GLFWwindow* window =
glfwCreateWindow(WINDOW_WIDTH,
WINDOW_HEIGHT, "Moving Car with
Rotating Wheels", NULL, NULL);
    if (!window) {
        glfwTerminate();
        return -1;
    }

    glfwMakeContextCurrent(window);

    // Register the mouse button callback
    glfwSetMouseButtonCallback(window,
mouseButtonCallback);

    // Main loop
    while
(!glfwWindowShouldClose(window)) {
        // Clear the window with white color
        glClear(GL_COLOR_BUFFER_BIT);
        glClearColor(1.0f, 1.0f, 1.0f, 1.0f);

        // Draw the moving car
        drawCar(carPosX);

        // Update car position and wheel
rotation only if the mouse is pressed
        if (isMousePressed) {
            carPosX += 0.0005f;  // Move car to
the right (slower speed)
            wheelAngle -= 1.0f;  // Rotate
wheels (slower)

            // Reset the car's position if it goes
off the screen
            if (carPosX > 1.2f) {
                carPosX = -1.2f;
            }
        }
```

```
    // Swap buffers and poll events
    glfwSwapBuffers(window);
    glfwPollEvents();
}
```

```
    glfwTerminate();
    return 0;
}
```